Dr. Jayasudha M Associate Professor Grade 2

CISM_HUMANIZED

Document Details

Submission ID
trn:oid:::3618:154880323

Submission Date
Oct 16, 2026, 4:49 AM GMT+0

Download Date
Oct 16, 2026, 4:52 AM GMT+0

File Name
CISM_HUMANIZED.pdf

File Size
1.2 MB

68 Pages

18,158 Words

103,530 Characters

71% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely revised by AI.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

129 AI-generated 71%
Likely generated or likely generated and revised by AI.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

A project report on

MEMORY OPTIMIZATION FOR SMALL-LANGUAGE-MODEL-DRIVEN SECURITY AGENTS

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering (Core)

by

MADHAN T B (23BCE1087)
MANIKANDAN S (23BCE1476)
SUSHINDH A (23BCE1513)



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

October, 2026

A project report on

MEMORY OPTIMIZATION FOR SMALL-LANGUAGE-MODEL-DRIVEN SECURITY AGENTS

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering (Core)

by

MADHAN T B (23BCE1087)
MANIKANDAN S (23BCE1476)
SUSHINDH A (23BCE1513)



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

October, 2026

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

DECLARATION

I hereby declare that the thesis entitled "**Memory optimization for small-language-model-driven security agents**" submitted by **[Name of the Candidate (Register Number)]**, for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Core), Vellore Institute of Technology, Chennai is a record of bonafide work carried out by me under the supervision of Dr. Jayasudha M.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai

Date: **Signature of the Candidate**

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the report entitled "**Memory optimization for small-language-model-driven security agents**" is prepared and submitted by **[Name of the Candidate (Register Number)]** to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology in Computer Science and Engineering (Core)** is a bonafide record carried out under my guidance. The project fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature of the Guide:

Name: Dr./Prof. Jayasudha M

Date:

Signature of the Examiner

Name:

Date:

Signature of the Examiner

Name:

Date:

Approved by the Head of Department,

B.Tech. Computer Science and Engineering (Core)

Name:

Date:

ABSTRACT

Vulnerability assessment and penetration testing (VAPT) is still an expert-driven, largely manual discipline. The tools themselves are mature and scriptable. What stays human is the judgement that sequences them, reads what a result means and keeps track of what has already been ruled out. Large language models look like a good fit for that orchestration layer, but the hosted models strong enough to do the job need target names, banners, credentials and raw tool output sent to a third party, and an engagement contract usually forbids that data from leaving the client.

This project presents CISM (Cyber Investigation State Memory), an autonomous VAPT agent driven by a small language model (SLM) served locally over an OpenAI-compatible endpoint, so no engagement data leaves the operator's machine. The agent is a LangGraph state machine with six nodes: think, act, extract, route, remember and respond. Its main contribution is the dependency and error propagation graph (DEPG). A per-tool extractor turns every successful tool invocation into nodes and edges, each carrying the quoted output that justifies it, so a claim the model cannot ground against real tool text is demoted instead of reported. A tiered, salience-ranked memory survives across turns. A six-phase PTES state machine constrains what may happen next, and a guardrail chain blocks out-of-scope targets, dangerous arguments and phase-illegal tools before any process is spawned. Nine real tools execute only through one sandboxed, argv-only seam behind a hard allowlist, and the operator works in a terminal interface that consumes a frozen event stream, which makes every run recordable and replayable.

CISM was evaluated against a containerized Damn Vulnerable Web Application on an isolated Docker network, driven by a locally hosted llama.cpp server, with the agent running inside the hardened container it ships with. The live runs exposed three extractor defects. Correcting them took one engagement from zero to eight evidence-backed findings across the affected tools. The headless harness reproduced a first finding in about three seconds over repeated runs.

Keywords: penetration testing, autonomous agents, small language models, local inference, dependency and error propagation graph, PTES, guardrails, sandboxing.

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to **Dr.Jayasudha M**, **Associate Professor**, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for his/her constant guidance, continual encouragement, understanding; more than all, he/she taught me patience in my endeavour. My association with him/her is not confined to academics only, but it is a great opportunity on my part of work with an intellectual and expert in the field of **autonomous systems and cyber security**.

It is with gratitude that I would like to extend my thanks to the visionary leader Dr. G. Viswanathan our Honorable Chancellor, Mr. Sankar Viswanathan, Dr. Sekar Viswanathan, Dr. G V Selvam Vice Presidents, Dr. Sandhya Pentareddy, Executive Director, Ms. Kadhambari S. Viswanathan, Assistant Vice-President, Dr. V. S. Kanchana Bhaaskaran Vice-Chancellor, Dr. T. Thyagarajan Pro-Vice Chancellor, VIT Chennai and Dr. P. K. Manoharan, Additional Registrar for providing an exceptional working environment and inspiring all of us during the tenure of the course.

Special mention to Dr. Viswanathan V, Dean, Dr. Nithyanandam P, Associate Dean, Dr. Suganya G, Associate Dean, Dr. Sweetlin Hemalatha C, Associate Dean, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai for spending their valuable time and efforts in sharing their knowledge and for helping us in every aspect.

In jubilant state, I express ingeniously my whole-hearted thanks to **Dr. Prassanna J**, Head of the Department, B.Tech. **Computer Science and Engineering (Core)** and the Project Coordinators for their valuable support and encouragement to take up and complete the thesis.

My sincere thanks to all the faculties and staffs at Vellore Institute of Technology, Chennai who helped me acquire the requisite knowledge. I would like to thank my parents for their support. It is indeed a pleasure to thank my friends who encouraged me to take up and complete this task.

Place: Chennai

Date: **[Name of the Candidate]**

CONTENTS

Chapter 1 Introduction 1
1.1 INTRODUCTION 1
1.2 OVERVIEW OF AUTONOMOUS PENETRATION TESTING 2
1.3 CHALLENGES IN SLM-DRIVEN PENETRATION TESTING 3
1.3.1 CONTEXT WINDOW AND STATE LOSS 3
1.3.2 UNGROUNDED FINDINGS 3
1.3.3 SCOPE, SAFETY AND BLAST RADIUS 3
1.3.4 AUDITABILITY AND REPRODUCIBILITY 4
1.3.5 CAPABILITY LIMITS OF SMALL MODELS 4
1.4 PROJECT STATEMENT 4
1.5 OBJECTIVES 5
1.6 SCOPE OF THE PROJECT 6
1.7 ORGANIZATION OF THE REPORT 6
Chapter 2 Background 7
2.1 INTRODUCTION 7
2.2 THE PENETRATION TESTING EXECUTION STANDARD 7
2.3 SURVEY ON MODEL-DRIVEN OFFENSIVE SECURITY AGENTS 7
2.4 SMALL LANGUAGE MODELS AND LOCAL INFERENCE 9
2.5 AGENT ORCHESTRATION FRAMEWORKS 9
2.6 DEPENDENCY AND ERROR PROPAGATION GRAPHS 10
2.7 SANDBOXING AND GUARDRAILS FOR TOOL-USING AGENTS 10
2.8 THREAT MODEL AND ETHICAL CONSIDERATIONS 11
2.9 SUMMARY OF GAPS IDENTIFIED 12
Chapter 3 Requirements and System Design 13
3.1 INTRODUCTION 13
3.2 FUNCTIONAL REQUIREMENTS 13
3.3 NON-FUNCTIONAL REQUIREMENTS 14
3.4 SYSTEM ARCHITECTURE 15
3.5 THE AGENT EXECUTION LOOP 16
3.6 PHASE MODEL 16
3.7 MODULE DESIGN 17
3.7.1 AGENT CORE 17
3.7.2 DEPENDENCY AND ERROR PROPAGATION GRAPH 18
3.7.3 MEMORY STORE 18
3.7.4 TOOL CATALOG AND SANDBOX 18
3.7.5 GUARDRAIL CHAIN 18
3.7.6 TERMINAL USER INTERFACE 19
3.7.7 REPORTING 19
3.8 THE EVENT CONTRACT 19
3.9 DESIGN DECISIONS AND TRADE-OFFS 20
3.10 HARDWARE AND SOFTWARE REQUIREMENTS 21

3.11 THE AGENT STATE OBJECT 22
Chapter 4 Implementation 23
4.1 INTRODUCTION 23
4.2 PROJECT ORGANIZATION 23
4.3 THE EXECUTION LOOP IN CODE 23
4.4 THE INFERENCE CLIENT AND BACKEND ABSTRACTION 25
4.5 PHASE STATE MACHINE AND PHASE ROUTING 26
4.6 TOOL CATALOG AND SANDBOXED EXECUTION 26
4.7 THE GUARDRAIL CHAIN 27
4.8 GRAPH CONSTRUCTION AND EXTRACTION 28
4.9 MEMORY 28
4.10 THE TERMINAL INTERFACE 29
4.11 CREDENTIAL REDACTION 30
4.12 OPERATIONAL LOGGING AND REPLAY 31
4.13 REPORT GENERATION 31
4.14 TESTING STRATEGY 31
4.15 LABORATORY ENVIRONMENT AND CONTAINER HARDENING 33
Chapter 5 Results and Discussion 34
5.1 INTRODUCTION 34
5.2 EXPERIMENTAL SETUP 34
5.3 BENCHMARKS: RENDERING STRATEGY AND REPRODUCIBILITY 36
5.4 LIVE ENGAGEMENT RESULTS 37
5.5 MEMORY EFFICIENCY AND CONTEXT HANDLING 39
5.6 WHAT FINE-TUNING WOULD BUY 40
5.7 REAL-WORLD USE CASES AND A CASE STUDY 41
5.7.1 CASE STUDY: THE DVWA ENGAGEMENT 41
5.7.2 WHERE THIS ARCHITECTURE FITS 41
5.8 LIMITATIONS 42
Chapter 6 Conclusion and Future Work 44
6.1 CONCLUSION 44
6.2 FUTURE WORK 46
REFERENCES 47
Appendices 52
Appendix 1 OPERATOR COMMANDS 52
Appendix 2 KEY BINDINGS 53
Appendix 3 REPRODUCING THE LABORATORY ENVIRONMENT 53
Appendix 4 DOMAIN MODEL REFERENCE 54
Appendix 5 TEST SUITE COVERAGE BY AREA 55
Appendix 6 PHASE TRANSITION TABLE 56
Appendix 7 INFERENCE SERVER CONFIGURATION 56

LIST OF TABLES

Table 3.1 Functional requirements 13
Table 3.2 Non-functional requirements 14
Table 3.3 PTES phases modelled by the agent 17
Table 3.4 The agent event vocabulary 19
Table 3.5 Principal design decisions 20
Table 3.6 Development and deployment environment 21
Table 3.7 Fields of the agent state object 22
Table 4.1 Package responsibilities 23
Table 4.2 The tool catalog 26
Table 4.3 Implemented guardrail stages 27
Table 5.1 Experimental configuration 35
Table 5.2 Streaming render cost for eight thousand tokens 36
Table 5.3 Resolved findings before and after the extraction fixes 37
Table 5.4 Timing profile of the live engagement 38
Table A.1 Operator commands 52
Table A.2 Key bindings 53
Table A.3 Fields of a graph node 54
Table A.4 Fields of a memory item 54
Table A.5 Coverage by area 55
Table A.6 Legal phase transitions 56
Table A.7 Inference server flags 57

LIST OF FIGURES

Figure 3.1 System architecture of CISM 15
Figure 3.2 The agent execution loop..... 16
Figure 3.3 Legal phase transitions 17
Figure 4.1 LangGraph node wiring and conditional routing 25
Figure 4.2 The terminal interface during a live engagement 29
Figure 4.3 The dependency and error propagation graph view 30
Figure 4.4 The memory view..... 30
Figure 4.5 Excerpt of a generated report 32
Figure 4.6 Laboratory network topology 33
Figure 5.1 Per-token render cost by streaming strategy 37
Figure 5.2 Graph produced by a live engagement against the laboratory target 38

LIST OF ACRONYMS

Acronym	Expansion
API	Application Programming Interface
CISM	Cyber Investigation State Memory
CLI	Command-Line Interface
CPU	Central Processing Unit
DEPG	Dependency and Error Propagation Graph
DVWA	Damn Vulnerable Web Application
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
IP	Internet Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
LoRA	Low-Rank Adaptation
MoE	Mixture of Experts
MSSP	Managed Security Service Provider
OS	Operating System
PTES	Penetration Testing Execution Standard
RAM	Random Access Memory
SLM	Small Language Model
SQL	Structured Query Language
SVG	Scalable Vector Graphics
TUI	Terminal User Interface
UI	User Interface
URL	Uniform Resource Locator
VAPT	Vulnerability Assessment and Penetration Testing
VRAM	Video Random Access Memory
YAML	YAML Ain't Markup Language

Chapter 1
Introduction

1.1 INTRODUCTION

A penetration test is a structured, authorized and time-boxed attempt to compromise a system the way a real adversary would, so the defects it finds can be fixed before someone less friendly finds them [1]. Vulnerability assessment and penetration testing (VAPT) covers both halves of that work. The assessment half enumerates and classifies weaknesses; the testing half tries to show that a given weakness can actually be exploited. The second half is what makes the first one usable, because a scanner finding nobody has validated is still only a hypothesis.

The individual instruments of the discipline are mature. Port and service discovery, web server scanning, technology fingerprinting, directory brute-forcing, injection testing and credential testing all have well-known, scriptable implementations. The layer above them has not been automated. Someone still has to choose which instrument to reach for given what the last one returned, judge whether a banner is worth pursuing, remember across hours of work that a hypothesis has already been eliminated, and notice when the engagement has moved from reconnaissance into exploitation. That work is judgement, and a human operator does it while holding most of the state in their head and in a scratch file. The defensive side of the same problem is also well studied, including machine-learning detection of SQL injection [2]–[5] and of intrusions more generally [6]–[8].

Large language models suit that layer, because the work is reading semi-structured text, picking an action and justifying it. The obstacle is confidentiality. The hosted frontier models that orchestrate tools best require target addresses, service banners, discovered credentials and raw tool output to travel to a third-party provider, and that is the material an engagement contract usually keeps inside the client's estate. An offensive-security agent that only runs against a hosted endpoint is therefore unusable for a large share of real engagements, however well it reasons. Language-model agents of this kind now have dedicated surveys [9], and penetration testing has become one of their most active security applications [10]–[16]. Privacy-preserving techniques for language models, such as differentially private text generation [17] and

differential privacy for clinical coding [18], address a neighbouring concern: they protect the data a model is trained or prompted with, not the act of sending engagement material to a hosted provider.

This project starts from the other end. CISM (Cyber Investigation State Memory) is an autonomous VAPT agent built around a small language model (SLM) served locally over an OpenAI-compatible endpoint, so inference is an ordinary process on the operator's own machine and no engagement data crosses a network boundary. A small model reasons less reliably than a hosted frontier model, so the system compensates with structure instead of scale. An explicit phase state machine constrains what may happen next. A graph of evidence-backed findings replaces the model's own recollection, a salience-ranked memory survives across turns, and a guardrail chain refuses actions the model should not have proposed. The model supplies direction; the machinery around it supplies memory, grounding and safety.

1.2 OVERVIEW OF AUTONOMOUS PENETRATION TESTING

An autonomous penetration-testing agent is a perception-decision-action loop over a security toolchain [19]. It perceives by reading the textual output of a tool it has just run, decides by selecting the next tool and its arguments, and acts by executing that tool against a target inside the agreed scope. It also has to remember, because a finding is usually only visible in combination with an earlier one. An open port on its own is nothing. An open port, plus a version banner, plus a known defect in that version, plus a successful authentication attempt is a finding, and those four observations may be minutes or hours apart.

The engagement itself is conventionally divided into phases. The Penetration Testing Execution Standard (PTES) [20] is the structure adopted in this project, reduced to the six phases the agent actually models: reconnaissance, enumeration, vulnerability discovery, exploitation, post-exploitation, and reporting. Phases earn their place for two independent reasons. They bound the tool set, since a credential-testing tool offered during reconnaissance invites the model to skip straight to intrusive activity. They also give the operator a legible notion of progress, which a free-running agent otherwise lacks entirely.

A realistic engagement does not proceed linearly. Compromising one host frequently restarts reconnaissance against a second host reachable only from the first. A phase model that enforces a strict sequence mis-models the work, while one that permits arbitrary jumps enforces nothing. The design adopted here is an explicit transition table: each phase declares the phases it may legally move to, which makes a realistic pivot expressible and an arbitrary jump impossible.

1.3 CHALLENGES IN SLM-DRIVEN PENETRATION TESTING

1.3.1 CONTEXT WINDOW AND STATE LOSS

A penetration test generates far more text than a small model can hold. A single full-port service scan can emit over a thousand lines, and a web scanner produces hundreds. A model with a thirty-two-thousand-token context window will have evicted the reconnaissance output long before exploitation begins. Appending every tool result to the conversation therefore guarantees that the agent forgets the evidence for its own conclusions at the moment it needs that evidence. State has to live outside the conversation, in a structure the agent can query [21], and the conversation has to carry references to it instead of raw output.

1.3.2 UNGROUNDED FINDINGS

A language model asked to summarize tool output will, under uncertainty, produce a fluent and plausible summary that the output does not support [22]. In a security report that failure mode does unusual damage. A hallucinated vulnerability consumes remediation effort the client pays for, and it discredits the real findings beside it. Every claim the agent records must therefore trace back to a literal span of real tool output, and a claim that cannot be traced has to be visibly demoted instead of quietly accepted.

1.3.3 SCOPE, SAFETY AND BLAST RADIUS

An agent that can spawn processes with network access and attacker-grade tooling is a liability unless its reach is constrained before execution. Three distinct hazards arise: targets outside the authorized scope, arguments that turn a legitimate tool into an arbitrary file-read or code-execution primitive, and tools invoked at a point in

the engagement where they are not yet appropriate. Each one has to be checked against the actual arguments of the call, and each one has to deny by default.

1.3.4 AUDITABILITY AND REPRODUCIBILITY

A penetration test is a deliverable, and a deliverable has to be defensible. An operator must be able to say what the agent did, in what order, on what evidence, and why it chose one branch over another. That requirement shapes the architecture as strongly as any functional one. It demands that every externally visible occurrence be an immutable, sequenced record instead of a transient user-interface update, so a run can be stored, replayed, diffed, and quoted in a report.

1.3.5 CAPABILITY LIMITS OF SMALL MODELS

A model small enough to run on a single consumer GPU is measurably weaker at tool selection than a hosted frontier model. Two such scenarios were observed here: answering a tool-offering turn with a question about information already supplied, and calling a tool with an un-substituted placeholder argument. A design that assumes the model is always correct leaves no room to recover when it isn't. The design adopted here treats the model as a fallible proposer whose proposals get validated, constrained, grounded and, where possible, recovered from.

1.4 PROJECT STATEMENT

To design and implement an autonomous vulnerability assessment and penetration testing agent that is driven by a locally hosted small language model, that keeps all engagement data on the operator's machine, that records every finding as an evidence-backed node in a dependency and error propagation graph instead of as model prose, that constrains its own actions through an explicit phase state machine and a guardrail chain evaluated before execution, that executes every real tool inside a sandbox through a single audited seam, and that presents the whole engagement to the operator through a terminal interface which only consumes a recordable and replayable event stream.

1.5 OBJECTIVES

1. To design an agent architecture in which reasoning, tool execution, finding extraction, phase progression and memory consolidation are separate, independently testable stages of one explicit state machine.
2. To implement a dependency and error propagation graph (DEPG) that converts raw tool output into typed, evidence-backed nodes and edges, and that derives each node's state from its evidence and the support of its preconditions.
3. To implement per-tool deterministic extractors for the tools whose output format is stable, with a grounded, structured-output model-based extractor as the fallback for the remainder, demoting a node whenever a claimed quotation is absent from the real output.
4. To implement a tiered, salience-ranked memory store that survives across turns and collapses recurring observations onto a single row instead of duplicating them.
5. To implement a six-phase PTES state machine with an explicit legal-transition table, and to gate tool availability on the current phase.
6. To implement a chain-of-responsibility guardrail evaluated before execution, carrying scope, dangerous-argument and phase-tool stages, with a tool registry that is a hard allowlist.
7. To execute every real tool through a single sandboxed seam with argv-only invocation, a disposable scratch directory, bubblewrap isolation and a process-tree-killing timeout.
8. To build a terminal user interface that consumes a frozen, sequenced event stream, so a run can be recorded, replayed and exported without the interface ever calling into the agent.
9. To validate the system against a live containerized vulnerable target driven by a locally hosted inference server, measuring findings produced, time to first finding, and rendering cost.

1.6 SCOPE OF THE PROJECT

The project delivers a working agent, its supporting domain model, its operator interface, a hardened container to run it in, a headless benchmark harness, and a test suite. The following fall within scope:

- A LangGraph-backed agent core with six executable nodes and a checkpointer that preserves state across turns within a process.
- A catalog of nine real security tools, eight of them declared declaratively and one hand-written, each reachable only through a sandboxed execution seam.
- An evidence-backed DEPG with deterministic extractors for the stable-format tools and a grounded model-based fallback for the rest.
- A terminal user interface with a scrolling transcript, a one-line adaptive status bar, and two modal views over the graph and the memory store.
- Markdown, JSON and SVG report generation from the final graph and memory state.
- Validation against a containerized Damn Vulnerable Web Application target on an isolated Docker network, driven by a local llama.cpp server.

1.7 ORGANIZATION OF THE REPORT

Chapter 2 sets out the background the design rests on: the phase model, the state of the art in model-driven offensive security, the constraints of local small-model inference, the graph formalism used to hold findings, the containment techniques applied to tool execution and the threat model the system assumes. Chapter 3 states the requirements and presents the design that satisfies them, from the layering down to the event contract joining the agent to its interface. Chapter 4 describes the implementation, covering the tool catalog, the sandbox seam, the guardrail stages, the extraction layer, the interface and the testing strategy. Chapter 5 reports what the system does when it runs. Chapter 6 concludes and sets out the ordered future work.

Chapter 2
Background

2.1 INTRODUCTION

Chapter 1 named the problems the design has to answer. This chapter supplies the material those answers rest on: the phase standard the agent follows, the published systems it is measured against, what a locally served model can and cannot be relied on to do, the orchestration framework chosen to express it, the graph formalism that holds its findings, and the containment techniques applied to tool execution. It closes with the threat model the system assumes and the gaps Chapter 3 answers.

2.2 THE PENETRATION TESTING EXECUTION STANDARD

PTES organizes an engagement into seven sections: pre-engagement interactions, intelligence gathering, threat modelling, vulnerability analysis, exploitation, post-exploitation and reporting [20]. For an autonomous agent the useful part of the standard is its ordering, which encodes the professional convention that intrusive actions follow enumeration, and enumeration follows discovery. An agent that ignores the ordering will, given an ambiguous instruction, reach straight for the most aggressive tool it has.

CISM models six phases: reconnaissance, enumeration, vulnerability discovery, exploitation, post-exploitation and reporting. Each one carries a system prompt telling the model what the phase is for, and the subset of the tool catalog legal in it. The phase therefore narrows the tool surface available to the model, and the model cannot ignore it.

2.3 SURVEY ON MODEL-DRIVEN OFFENSIVE SECURITY AGENTS

Work on applying language models to offensive security falls into three broad groups. The first treats the model as an advisor. The operator runs the tools, and the model interprets output and suggests a next step. This arrangement carries the least risk and the least leverage, since the human stays in every loop [10].

The second group treats the model as an orchestrator with shell access. A prompt loop gets a terminal and a goal, and each command's output feeds back as the next observation. Published systems in this group show that a strong enough hosted model can chain several steps of a simple engagement unaided [11]–[13]. They also share three weaknesses that bear directly on this work. Depending on a hosted frontier model makes them unusable wherever engagement data cannot leave the client estate. Unrestricted shell access leaves the blast radius of a mis-generated command unbounded. And their findings live only as prose in a transcript, so nothing separates an observation the tool output supports from one the model inferred. Later systems extend the pattern to multi-agent coordination with domain-adapted models [14] and to web-application targets [15], [16].

The third group focuses on specific sub-tasks, such as generating an exploit for a known defect, writing a scanner rule, or triaging a crash. This work is largely orthogonal to orchestration. It matters here mainly as evidence that model-generated security artefacts need verification against ground truth before use, which is the conclusion the grounded-extraction design arrives at from the orchestration side. Benchmarks for the underlying capability have appeared alongside these systems, scoring a model against self-contained security tasks instead of a whole engagement [23].

Cutting across those three groups is a question of how much of the engagement the model is trusted to decide. At one extreme the model picks the next command freely from an unrestricted shell. At the other it selects from a fixed menu of parameterized actions. The free-form arrangement looks more capable on paper and is markedly harder to contain, because the set of things a shell can be asked to do is unbounded and cannot be enumerated in advance for checking. The menu arrangement bounds the blast radius at the cost of confining the agent to whatever its author anticipated. This design sits deliberately on the menu side of that line, with the catalog declared as data so widening the menu stays a routine change.

A second difference is where the engagement's state is held. Most published systems hold it in the conversation itself, which couples the agent's memory to the model's context window and loses the oldest evidence first. A smaller number maintain an external store, usually a flat list of observations. Neither arrangement records the

dependency between observations, so neither can answer the question an operator actually asks at the end of a test, which is what the conclusions rest on. That question is what motivated the graph structure adopted here.

The common gap across the literature is structural. The agent's beliefs are never separated from its narration. A finding that exists only as a sentence in a transcript cannot be contradicted by later evidence, cannot be pruned when its precondition is disproved, and cannot be cited in a report. Recording findings as a graph with attached evidence, described in section 2.6, answers that gap.

2.4 SMALL LANGUAGE MODELS AND LOCAL INFERENCE

A small language model, for this work, is one that runs usefully on a single consumer accelerator: roughly one to eight billion active parameters, quantized [24]. Servers such as llama.cpp's server, vLLM, LM Studio and Ollama expose such models behind an OpenAI-compatible HTTP interface. An agent written against that interface can treat the model as a replaceable component, and can point at a hosted endpoint instead when policy allows it.

Local inference buys confidentiality and costs capability. Three consequences shape the design. Native tool-calling support varies across servers and quantizations, so the agent has to fall back on parsing a structured tool call out of plain text. Context windows are smaller, which pushes state outside the conversation [21]. Instruction-following degrades over long contexts, so the system has to treat a malformed or absent tool call as an expected event.

2.5 AGENT ORCHESTRATION FRAMEWORKS

An agent of this kind can be written as a hand-rolled loop interleaving reasoning and action in the prompt [19], as a graph of nodes with typed shared state, or as a set of cooperating agents [9]. The hand-rolled loop is the simplest to start and the hardest to test, because streaming, tool dispatch, state mutation and phase progression all end up interleaved in one function. The multi-agent arrangement adds a coordination problem this task does not have.

This project uses LangGraph, which expresses the agent as a directed graph of nodes over a typed state object, with a checkpointer that persists state between

invocations. The property that earns its place is that each node is an ordinary asynchronous function of state. The phase router, the extraction step and the memory consolidation step can each be tested in isolation with a constructed state and no model, no network and no tools. Streaming works at the node level, which lets the interface show narration as it is produced while knowing nothing about the graph.

2.6 DEPENDENCY AND ERROR PROPAGATION GRAPHS

A dependency and error propagation graph is a directed graph in which a node is a claim about the target and an edge is a dependency: the claim at the head holds its meaning only while the claim at the tail holds. The formalism comes from fault analysis, where it describes how the failure of a component invalidates everything downstream of it, and it transfers to security assessment with little modification, because a penetration test is largely an exercise in conditional claims. A credential is useful only while the service that accepts it stays reachable. An exploit is relevant only while the version it targets is the version actually deployed. Bayesian networks have been applied to network vulnerability assessment [25], and attack graphs support probabilistic inference about how compromise propagates through a network [26]. Such graphs can be generated automatically from a model of hosts and exploits [27], which applies the same conditional structure at network scale.

Three properties make the structure worth its implementation cost. A claim can be contradicted: when later evidence disproves a node, everything that depended on it is invalidated automatically instead of rotting in a transcript. A claim can be justified, because the evidence attached to a node records which tool produced which text at what time, which is what a report has to cite. And a claim can be deduplicated, since node identity derives from a normalized target and a cause class, so the same finding arriving from two tools collapses onto one node and the finding count stays honest.

2.7 SANDBOXING AND GUARDRAILS FOR TOOL-USING AGENTS

Containment for a tool-using agent operates at two levels. The outer level is operating-system isolation of the process the tool runs in: a restricted filesystem view, dropped capabilities, a constrained system-call surface, resource ceilings, and a timeout that kills the whole process tree and not just the leaf. Bubblewrap provides a usable

unprivileged implementation of the filesystem and namespace part, and container runtimes supply the rest [28], whose own weaknesses are well documented [29].

The inner level is argument-level policy, evaluated before the process exists. Operating-system isolation cannot tell a scan of an authorized host from a scan of the client's production network, since both are ordinary outbound connections. Only a check against the declared engagement scope can. The two levels complement each other and neither substitutes for the other: the inner one encodes the engagement contract, and the outer one limits the damage when the inner one gets it wrong.

2.8 THREAT MODEL AND ETHICAL CONSIDERATIONS

An offensive-security agent is a dual-use artefact, so the design has to say plainly who it is for and what it refuses to do. The system assumes an authorized operator running a test for which written permission exists, against targets enumerated in that permission. Every mechanism in this report follows from that assumption. The engagement scope is an allowlist, so a target nobody authorized is refused by default and not permitted until someone thinks to forbid it. The tool registry is a hard allowlist, so a tool nobody deliberately constructed into the run cannot be reached by name. The phase model withholds intrusive tooling until the engagement has actually progressed. Each one exists to make the unauthorized action the one that takes deliberate effort.

The design considers three adversaries. The first is the operator's own mistake: a mistyped address, a scope widened and never narrowed again, a tool invoked earlier than intended. The guardrail chain covers this class, because it evaluates the actual arguments of a call before a process exists. The second is the model, treated as an unreliable proposer. It may propose a target outside scope, an argument that reads a file off the host, or a tool the phase does not permit, and a stage that never consults the model refuses each of those. The third is the target, which has to be treated as hostile input: tool output is attacker-influenced text, and a model that reads it can be instructed by it. This is indirect prompt injection, demonstrated against deployed model-integrated applications [30] and the subject of a growing body of attack and defence work [31]–[34]. For that third case the tool-output seam carries a single screening stage that runs before tool output re-enters the prompt; defenses against injection aimed at the graph and the memory store are not yet implemented, which Chapter 5 records as a limitation and Chapter 6 carries as future work.

Two practical obligations follow for anyone running the system. The engagement scope has to be declared from the authorization document and not from memory, and the credential wordlists the credential-testing tool draws on may hold only lists the operator is permitted to use. The system ships neither on purpose, and the credential tool's arguments select one of two fixed, named lists instead of an arbitrary path, so the tool cannot be redirected into reading host files. Credentials leaked in public code repositories are a documented source of the material such lists contain [35].

2.9 SUMMARY OF GAPS IDENTIFIED

1. Published model-driven offensive agents assume a hosted frontier model, which the confidentiality constraints of most real engagements rule out.
2. Findings are recorded as transcript prose instead of structured, evidence-backed claims, so they cannot be contradicted, pruned, deduplicated or cited.
3. Containment is usually a container boundary alone, with no argument-level policy expressing the engagement scope and no denial of dangerous arguments before execution.
4. Phase progression, where it is modelled at all, is either absent or a strict linear sequence that mis-models realistic pivots.
5. Runs are not reproducible. Without an immutable event record, an engagement cannot be replayed, diffed or defended after the fact.
6. Engagement state is held in the conversation, or at best in a flat list of observations, so the agent’s memory is tied to the context window and nothing records the dependency between observations.
7. Tool-calling support varies across local inference servers and quantizations, so a design that assumes a well-formed native tool call is fragile; a malformed or absent call has to be an expected event.
8. Tool output re-enters the prompt as attacker-influenced text, so indirect prompt injection has to be treated as a design-level threat, with output screened before it is reintroduced.

Chapter 3

Requirements and System Design

3.1 INTRODUCTION

This chapter states the requirements the system was built against and the design that satisfies them. The design appears top-down: the architecture and its layers, the execution loop that drives a turn, the module responsibilities, and finally the event contract joining the agent to its interface.

3.2 FUNCTIONAL REQUIREMENTS

Table 3.1 Functional requirements

ID	Requirement
FR-1	The agent shall accept a natural-language instruction from the operator and respond with narration streamed as it is produced.
FR-2	The agent shall select and invoke security tools autonomously, subject to the guardrail chain and the current phase.
FR-3	Every tool invocation shall be checked against the declared engagement scope before the process is created, and refused if the target lies outside it.
FR-4	Every tool invocation shall be checked for arguments that permit filesystem access outside the scratch directory or arbitrary code execution, and refused if any are present.
FR-5	Every tool invocation shall be checked against the set of tools legal in the current phase, and refused if the tool is not offered by that phase.
FR-6	Successful tool results shall be converted into DEPG nodes and edges by a per-tool extractor, with a grounded model-based fallback where no deterministic extractor exists.
FR-7	A node whose supporting quotation cannot be located in the real tool output shall be demoted to the lowest confidence tier.
FR-8	The graph shall derive each node's state from its evidence and the support of its preconditions, and shall prune regions whose preconditions are disproved.
FR-9	The agent shall maintain a tiered, salience-ranked memory across turns, collapsing recurring observations onto a single row.

ID	Requirement
FR-10	Phase transitions shall occur only through the phase state machine and only along legal transitions; the operator may request a transition but may not force one from the interface.
FR-11	The operator shall be able to inspect the graph and the memory store at any time without interrupting the run.
FR-12	Credential-shaped input shall be masked before it reaches the transcript, the command history or an export, and the command carrying it shall be refused instead of forwarded.
FR-13	The agent shall emit an immutable, sequenced event for every externally visible occurrence, sufficient to replay the run.
FR-14	The system shall produce a Markdown report, a JSON report and a graph diagram from the final graph and memory state.
FR-15	The operator shall be able to extend or shrink the engagement scope during a run, and the change shall be reported as an event.

3.3 NON-FUNCTIONAL REQUIREMENTS

Table 3.2 Non-functional requirements

ID	Category	Requirement
NFR-1	Confidentiality	No engagement data shall be transmitted to any endpoint other than the operator-configured inference server and the targets within scope.
NFR-2	Security	The tool registry shall be a hard allowlist; a name not explicitly constructed into it shall raise instead of resolving.
NFR-3	Security	Tools shall be executed argv-only; no invocation shall pass through a shell.
NFR-4	Reliability	A tool failure shall reach the model as an observation, and shall not be raised as an exception that ends the turn.
NFR-5	Performance	Streaming narration shall have a per-token rendering cost that does not grow with the length of the entry.
NFR-6	Usability	The status line shall remain one line at any terminal width, degrading by priority instead of wrapping.
NFR-7	Maintainability	The interface shall never call into the agent; the only coupling shall be the event queue.
NFR-8	Testability	Every node, guardrail stage and extractor shall be testable without a model, a network or a live tool.

ID	Category	Requirement
NFR-9	Auditability	Graph steps, tool calls and guardrail decisions shall be written to a structured operational log with a retention policy.
NFR-10	Portability	The system shall run against any OpenAI-compatible inference server without code changes.

3.4 SYSTEM ARCHITECTURE

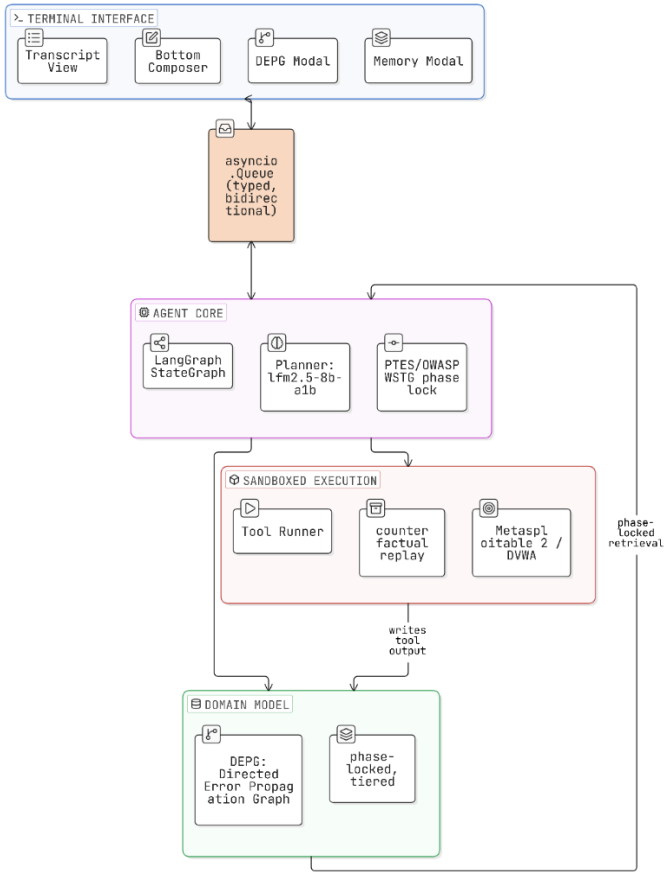


Figure 3.1 System architecture of CISM

The system has four layers. The interface layer is a terminal application that consumes events and owns no engagement state of its own. The agent layer holds the LangGraph state machine, the phase model, the guardrail chain and the tool registry. The domain layer holds the graph and the memory store, which are plain data structures that know nothing about models or terminals. The execution layer is the sandbox seam and the tools it runs. Dependencies point one way only: the interface depends on the event vocabulary, the agent depends on the domain and execution layers, and the domain layer depends on nothing but the event vocabulary it serializes into.

The heaviest decision in this layering is that the interface only consumes. The agent pushes immutable events onto an asynchronous queue, and the application drains that queue and mutates widgets. Nothing in the interface package calls into the agent, and no widget calls a model. A recorded log and a live agent therefore look identical to the interface, which is what makes the demonstration mode, the replay facility and the snapshot tests possible at all.

3.5 THE AGENT EXECUTION LOOP

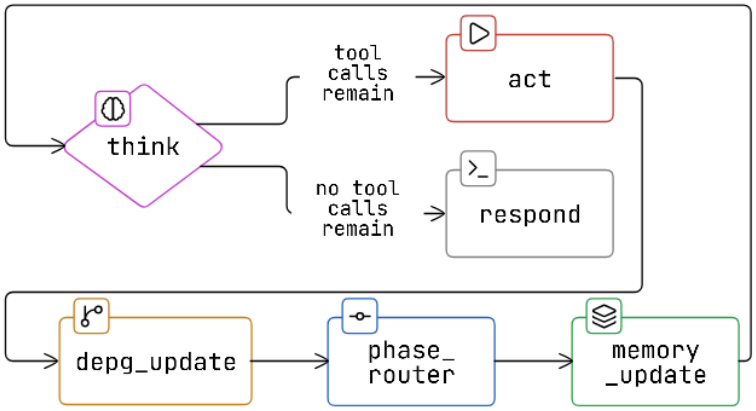


Figure 3.2 The agent execution loop

One turn is a cycle through six nodes. The think node binds the tools legal in the current phase and streams a reply; if the model produced tool calls and the per-turn cap has room, control passes to the act node, and otherwise to the respond node. The act node runs each proposed call through the action guardrail chain and then executes it, converting a failure into an observation instead of an exception. The extraction node turns each successful result into graph proposals. The phase router proposes a legal phase move, but only once the graph holds real evidence for the current phase. The memory node folds the step's findings, failures and narration into the memory store, and control returns to the think node.

3.6 PHASE MODEL

The six phases and the tools each one offers appear below. Legality is declared per tool in the catalog, so adding a tool to a phase is a data change and not a code change.

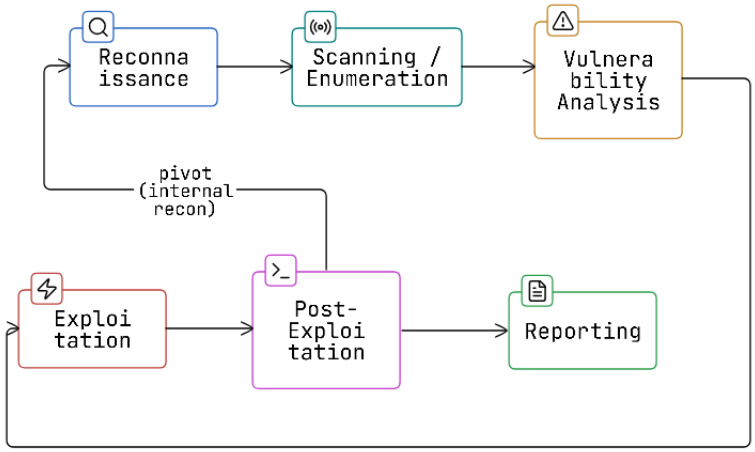


Figure 3.3 Legal phase transitions

Table 3.3 PTES phases modelled by the agent

Phase	Purpose	Representative tools
Reconnaissance	Establish what exists and what is reachable	nmap, curl, dnsenum
Enumeration	Establish what each reachable service is	whatweb, gobuster, nikto, wafw00f
Vulnerability discovery	Establish which weaknesses are plausibly present	nikto, sqlmap, wafw00f
Exploitation	Demonstrate that a weakness is real	sqlmap, hydra
Post-exploitation	Establish the consequence of a demonstrated weakness	curl, nmap
Reporting	Consolidate findings into a deliverable	(no tools; report generation)

3.7 MODULE DESIGN

3.7.1 AGENT CORE

The agent core owns the compiled graph, the checkpointer that keys state to a stable thread identifier, the translation of streamed model output into events, and the operator commands the interface cannot answer by itself. It is the only component that

satisfies the agent protocol the interface is written against, which is a queue and four methods: start, submit, interrupt and close.

3.7.2 DEPENDENCY AND ERROR PROPAGATION GRAPH

The domain model is a mutable node and edge pair plus a container. The container merges proposals as they arrive, resolves each dirty node's state from its evidence and the support of its preconditions, and prunes regions that are fully disproved. Node identity derives from a normalized target and a cause class, so a host written three different ways collapses onto one node family instead of fragmenting into three.

3.7.3 MEMORY STORE

The memory store holds tiered items. Each one carries the phase in which it was seen, a snippet, a hit count and a salience score computed from a tier weight, a time decay and a per-hit bonus. A graph-derived row keys on the node identifier. A narration or failure row keys on a digest of its source, phase and snippet, so a recurring observation increments a counter instead of producing a new row.

3.7.4 TOOL CATALOG AND SANDBOX

Eight tools are declared in a catalog file as data, namely binary, capability, target flag, timeout and legal phases. The ninth, the credential-testing tool, is hand-written, because it needs a scratch directory pre-populated with wordlists before the process starts. All nine are assembled into a registry that resolves a name to a bound tool and raises on an unknown name. Every real tool reaches the operating system through one seam, which creates a per-call scratch directory, invokes the binary argv-only under bubblewrap, caps captured output, and enforces a timeout that kills the process tree.

3.7.5 GUARDRAIL CHAIN

Guardrails are stages in a chain of responsibility with four seams: input, action, output and tool output. The action seam carries the scope, dangerous-argument and phase-tool stages and short-circuits on the first denial, and each of the other three seams carries a single stage. Adding a check means appending a stage; neither the chain runner nor the graph shape changes.

3.7.6 TERMINAL USER INTERFACE

The interface is a single scrolling transcript of per-entry widgets, a fixed composer, and a one-line status bar, with two modal overlays for the graph and the memory store. Styling lives entirely in a stylesheet, and no widget carries inline style. The transcript streams into a buffer the widget owns and refreshes on a coalescing timer, which keeps the per-token cost flat as an entry grows.

3.7.7 REPORTING

Report generation walks the final graph and memory state and emits a human-readable Markdown write-up, a structured JSON document for downstream tooling, and a graph diagram. It depends only on the shared event vocabulary and not on the agent package, so a report can be built from an event stream alone.

3.8 THE EVENT CONTRACT

Every externally visible occurrence is a frozen record carrying a monotonically increasing sequence number and a wall-clock timestamp. That contract is what makes a run recordable, replayable and diffable, and it is also how secrets stay out of the deliverable. The session-state event carries the name of the environment variable holding the key and a boolean saying whether it is set, and no field anywhere in the vocabulary could hold a key.

Table 3.4 The agent event vocabulary

Event	**Carries**	**Effect on the interface**
RunStarted	run identifier, phase, framework	starts the working indicator
Narration	entry identifier, text, done flag	streams into one transcript entry
ToolStarted	call identifier, tool, arguments	appends a tool entry
ToolOutput	call identifier, chunk	appends to that entry's collapsed body
ToolFinished	call identifier, summary, lines, tier, score	fills the summary line
ToolFailed	call identifier, reason, traceback	marks the entry failed; traceback stays collapsed

Event	Carries	Effect on the interface
PhaseChanged	phase, framework	draws a phase rule
ScopeChanged	action, target	reports a scope change that actually took effect
NarrationRedacted	entry identifier, reason	retroactively withholds a streamed entry
DEPGUpdated	nodes, edges	replaces the graph model
MemoryUpdated	rows, terms	replaces the memory model
TokenDelta	delta, total, context used, context maximum	updates the status bar
SessionState	model, endpoint, key variable name, key presence, target	updates model and target segments
Interrupted	reason	stops the indicator, marks the transcript
RunFinished	run identifier, success flag	stops the indicator

3.9 DESIGN DECISIONS AND TRADE-OFFS

Several decisions in this design had defensible alternatives. The reasoning behind each one is recorded below, so the cost of the choice stays visible.

Table 3.5 Principal design decisions

Decision	Alternative considered	Rationale
Locally hosted small model	Hosted frontier model	Engagement data must not leave the operator's estate; capability is recovered through the surrounding structure instead of model scale
Graph of evidence-backed findings	Findings as transcript prose	A prose finding cannot be contradicted, pruned, deduplicated or cited; a node with attached evidence can
Explicit transition table for phases	Strict linear sequence, or no phase model	A linear sequence mis-models a realistic pivot; no model at all leaves the aggressive tools available from the first turn
Guardrails evaluated before execution	Post-hoc filtering of results	A scan of an unauthorized host cannot be undone after the packets are sent

Decision	Alternative considered	Rationale
Interface as a pure event consumer	Interface calls the agent directly	A recorded log and a live agent look identical to it, which makes replay, export and snapshot testing possible
Declarative tool catalog	A Python function per tool	Adding a flag-and-target-shaped tool becomes a data change; the hand-written exception is justified by its scratch-directory requirement
Terminal as the only surface	A web dashboard	The operator already works in a terminal; a second surface would duplicate state and widen the attack surface for no operational gain
Buffered static transcript	Append-only log widget	An append-only widget cannot host an interactive toggle, so collapsed tool output would be impossible; the cost of the alternative was measured first

3.10 HARDWARE AND SOFTWARE REQUIREMENTS

Table 3.6 Development and deployment environment

Component	Requirement
Processor	x86-64, 4 cores or more
Memory	16 GB RAM; 2 GB ceiling applied to the agent container
Accelerator	A consumer GPU sufficient to serve a quantized small model; CPU-only inference is supported but slower
Operating system	Linux, or Windows with a Linux subsystem; bubblewrap is required for live tools
Language runtime	Python 3.11 or later
Core libraries	LangGraph and LangChain for orchestration, Textual for the terminal interface, PyYAML for the tool catalog, Graphviz for diagram rendering
Inference server	Any OpenAI-compatible server: llama.cpp, vLLM, LM Studio, or Ollama's compatible endpoint
Security tools	nmap, curl, nikto, whatweb, dnsenum, hydra, sqlmap, gobuster, wafw00f
Containerization	Docker, with a user-defined bridge network for the isolated laboratory
Testing	pytest with parallel execution and terminal snapshot support

3.11 THE AGENT STATE OBJECT

Every node in the graph is a function of one typed state object, and that object is the complete description of an engagement in progress. Keeping it explicit is what lets a node be tested with a constructed state and no model, no network and no tools, and it is also what the checkpointer persists between turns.

Table 3.7 Fields of the agent state object

Field	**Holds**
messages	The conversation so far, appended to and never replaced, so a node never loses an earlier turn
phase	The current PTES phase, which determines the system prompt and the tools bound to the model
phase changed	Whether this step moved the phase, so that the interface can draw a phase rule exactly once
graph nodes	The findings proposed so far, keyed by the identity derived from target and cause class
graph edges	The dependencies between those findings
memory items	The tiered, salience-ranked rows retained across turns
run identifier	The identifier every event and log line for this run carries
target	The declared target environment, where the operator has supplied one
context window	The real ceiling discovered from the inference server, reported by the status gauge
compacted	How many older messages the think node dropped to fit the context window, zero when nothing was dropped

The last field makes a silent failure visible. When a conversation outgrows the context window something has to be discarded, and an agent that discards quietly appears to forget for no reason. Recording how much was dropped turns that into an observable quantity.

Chapter 4
Implementation

4.1 INTRODUCTION

This chapter describes how the design of Chapter 3 was realized. It follows the same order, covering project organization, the execution loop, the phase machine, tools and sandboxing, guardrails, extraction, memory, the interface, logging and reporting, and it closes with the laboratory environment and the container hardening applied to the agent itself.

4.2 PROJECT ORGANIZATION

Table 4.1 Package responsibilities

Package	Responsibility
agent	The LangGraph wiring, the typed state, the phase profiles, the guardrail chain and the inference client
agent.tools	The declarative tool catalog, the registry allowlist, the sandbox seam and the offline stand-ins
agent.guardrails	The chain runner and the scope, dangerous-argument and phase-tool stages
depg	The graph domain model: nodes, edges, evidence, the container, the state reducer, the extractors and the identity transforms
memory	The tiered, salience-ranked memory store and its items
tui	The terminal application, the event vocabulary, redaction, replay and the widgets
report	Markdown, JSON and diagram generation from final state
tests	A test module per source module, plus terminal snapshot tests

4.3 THE EXECUTION LOOP IN CODE

The graph is assembled by a single factory function that wires the six steps onto a typed state object and compiles the result against a checkpointer; every dependency the steps need (the chat model, the tool registry, the guardrail chains, the extractor registry and the phase profiles) is passed in rather than constructed internally, so the assembly step itself is identical whether what sits behind it is a dummy model and

dummy tools or a real local inference server and a live sandbox. Figure 4.1 shows the wiring exactly as it is written: a conditional branch after the reasoning step routes to execution when the model produced tool calls and the turn's per-step cap still has room, and to the final response otherwise; the four remaining steps (execution, graph extraction, phase routing and memory consolidation) form a fixed sequence that always runs together and always returns control to the reasoning step, so a single turn is one pass around that lower loop and a whole engagement is many such passes chained by the checkpointer.

The reasoning step binds the tool subset the current phase profile exposes, screens any prior tool-failure snippet through the tool-output guardrail chain before it re-enters the prompt, and streams the model's reply token by token, accumulating the stream into one message rather than forwarding each chunk as a separate turn. Native tool-calling support varies across local inference servers, so when the phase offers tools and the streamed reply produced none through the server's native mechanism, the step falls back to parsing a structured tool-call block out of the plain-text reply before deciding there were none. The input guardrail chain has already run on the operator's own submission earlier in the turn, so this step's only input-side responsibility is screening tool output already in the conversation, not the operator's text.

The execution step is the only place a tool is run. For each proposed call it first runs the action guardrail chain. That chain starts with the current phase's own tool guard, built from the exact set of tools the same profile bound to the reasoning step so the re-check cannot drift from what the model was offered, and continues with the scope guard, the dangerous-flag denylist and, when configured, a privacy swap-out stage. Only a call every stage allows reaches the matching entry in the tool registry. A blocked or failed call becomes a tool message carrying the failure text instead of raising, so the model observes the refusal or the error on its next turn and can react to it rather than the run ending in an exception. A real tool call is a blocking subprocess under the sandbox, so the step awaits it rather than running it synchronously, which keeps a slow tool from blocking the event loop the rest of the interface runs on.

State persists across turns within a process through the checkpointer, an in-process store keyed by a stable per-agent identifier, so a second instruction sees the graph, the memory and the phase the first one produced. Section 3.11 sets out the full

field list of that state object; what matters here is that every one of those fields is plain data a step reads and returns a partial update for, never an object a step mutates in place, which is what lets a step be unit-tested against a constructed state with no model, no network and no tools, and the same property that makes a run replayable from its recorded event log. The checkpointer does not survive a process restart, a deliberate limitation Chapter 6 returns to.

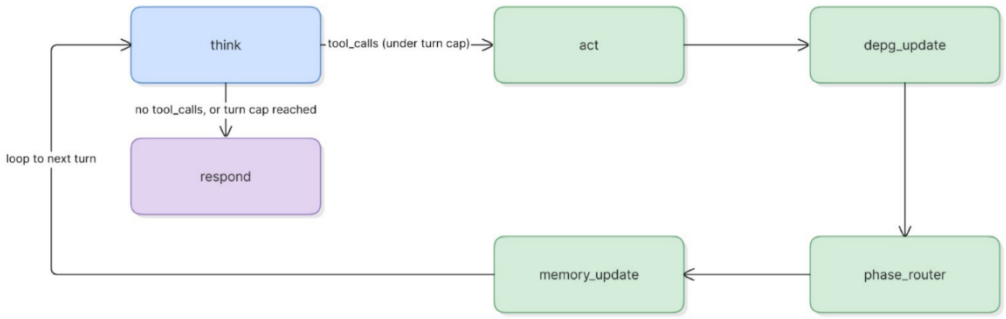


Figure 4.1 LangGraph node wiring and conditional routing

4.4 THE INFERENCE CLIENT AND BACKEND ABSTRACTION

The inference client is thin on purpose, because the model is the component most likely to be replaced. A single factory builds a chat model from a base address, a model name and the name of an environment variable holding the key, and returns a canned deterministic model when nothing is configured. The whole system therefore runs, and the whole test suite passes, with no server, no weights and no network. That canned model is the default production path until an operator points the agent at a real endpoint, so the offline path is exercised constantly.

Two differences between servers had to be absorbed. Context-window discovery is not standardized across OpenAI-compatible servers, so the client probes where a server exposes the information and falls back to a declared default where it does not; the discovered value is what the status bar's context gauge reports, so an operator sees the real ceiling. Structured extraction is the second difference. A generic function-calling path works on any compliant server, while a raw completion endpoint with a constrained output schema is available for servers that support it, selected by a command-line option. The extraction backend is therefore an interface with two implementations instead of a hard dependency on one wire format.

4.5 PHASE STATE MACHINE AND PHASE ROUTING

The phase enumeration and its legal-transition table live in one module, and every transition passes through a single method on the state machine, which gives the rest of the system one choke point to rely on. The phase router advances the phase only once the graph holds real evidence for the current phase. An earlier implementation advanced one phase per successful tool call, which let an engagement reach exploitation on the strength of a scan that found nothing, and the audit corrected it.

Phase legality is enforced twice, on purpose. The phase profile binds only the legal tools to the model, so an illegal tool is normally never offered. A guardrail stage then re-checks the tool against the phase at call time, so a call constructed from a stale binding or from parsed text is still refused.

4.6 TOOL CATALOG AND SANDBOXED EXECUTION

Eight tools are rows in a catalog file. A row declares the binary, the capability it provides, the flag that carries the target, a timeout and the phases in which it is legal, so extending the catalog with another flag-and-target-shaped tool is a data change. The credential-testing tool does not fit that flat shape, because it needs a scratch directory pre-populated with credential files before the process starts, so it is written by hand next to the catalog. Its wordlist arguments select one of two fixed, named lists and never an arbitrary path, which stops a compromised model turning a credential tool into a way to read files from the host.

Table 4.2 The tool catalog

Tool	**Capability**	**Legal phases**
nmap	Host and service discovery	Reconnaissance, post-exploitation
curl	Direct request issuing	Reconnaissance, post-exploitation
dnsenum	Domain enumeration	Reconnaissance
whatweb	Technology fingerprinting	Enumeration
gobuster	Content and directory discovery	Enumeration
nikto	Web server scanning	Enumeration, vulnerability discovery

Tool	Capability	Legal phases
wafw00f	Web application firewall detection	Enumeration, vulnerability discovery
sqlmap	Injection testing	Vulnerability discovery, exploitation
hydra	Credential testing (hand-written)	Exploitation

Every real tool reaches the operating system through a single function. It creates a scratch directory that no longer exists when the call returns, invokes the binary with an argument vector and never through a shell, runs it under bubblewrap with a restricted filesystem view, caps the captured output, and enforces a timeout that kills the entire process tree and not only the leaf process. The seam sits behind a runner protocol, so a container-backed implementation can replace the subprocess implementation without any tool function changing.

4.7 THE GUARDRAIL CHAIN

Table 4.3 Implemented guardrail stages

Stage	Seam	What it refuses
Scope guard	Action	A call whose target falls outside the declared engagement scope, which is an allowlist of networks and domains with a denylist checked first
Dangerous argument guard	Action	A call whose free-form arguments could read or write outside the scratch directory, or trigger script execution such as the scripting engine of the port scanner
Phase tool guard	Action	A call whose tool is not offered by the current phase, re-checked at call time and never trusted from the binding

The scope guard was revised after review. An early version of the command-line option meant to exclude a target folded it into the allowlist, so a flag named exclude only ever widened the permitted set, which is the opposite of what an operator would expect from a scope guard. The scope object now carries a real denylist consulted before the allowlist, and the option builds into that denylist.

All three stages are heuristic safety nets and not formal guarantees, since they can only inspect what a call's arguments actually contain. This limitation is recorded explicitly, and replacing the argument denylist with an allowlist grammar is identified as future work.

4.8 GRAPH CONSTRUCTION AND EXTRACTION

Extraction is the seam through which a tool result becomes graph proposals. Tools whose output format is stable have deterministic extractors that parse their own text directly and assign a deterministic cause class. Every other tool falls back to a grounded model-based extractor, which requests structured output and is clamped to the lowest confidence tier when a quotation it claims is absent from the tool's real output. Failing that, a generic extractor stamps one untested node per call carrying the tool's own name as the cause class.

Three defects in this layer surfaced only when the system ran against a real target, and they show why live validation was necessary. The content-discovery extractor matched none of five real hits, one of them an exposed installation script, because the real output interleaves a carriage return and terminal escape sequences from a progress indicator, and those defeated a regular expression written against clean sample text. The fingerprinting tool had no extractor at all and degraded to an opaque hash signal. The firewall-detection tool defaulted to a secure scheme and failed outright against a plaintext-only target. All three were fixed, and the regression tests for the fixes were built from the actual captured output instead of idealized samples.

Node identity is computed after normalization. Scheme, port, case and a trailing dot are stripped from the target before it becomes part of the identity, so three spellings of the same host collapse onto one node family. Each new node is linked to the node that led to it, and the container resolves state from evidence and parent support, pruning regions whose preconditions are disproved.

4.9 MEMORY

The memory node folds each step's findings, failures, narration and phase changes into the store. A graph-derived row keys on the node identifier, so the row tracks the node. A narration or failure row keys on a digest of its source, phase and

snippet, so a recurring message increments a hit counter instead of producing a new row. Salience combines the tier weight, a time decay and a small per-hit bonus, using the same coefficients as the recorded demonstration log, so a live run and a replay decay identically.

4.10 THE TERMINAL INTERFACE

The transcript is built from per-entry widgets inside a scrolling container, with the composer and status bar described in section 3.7.6. Two implementation choices matter here. An append-only log widget was rejected as the transcript surface, because such a widget cannot host an interactive toggle, which would have ruled out collapsed tool output. The shipped widget instead appends into a buffer it owns and repaints on a coalescing timer, and section 5.4 reports what that costs against the two alternatives.

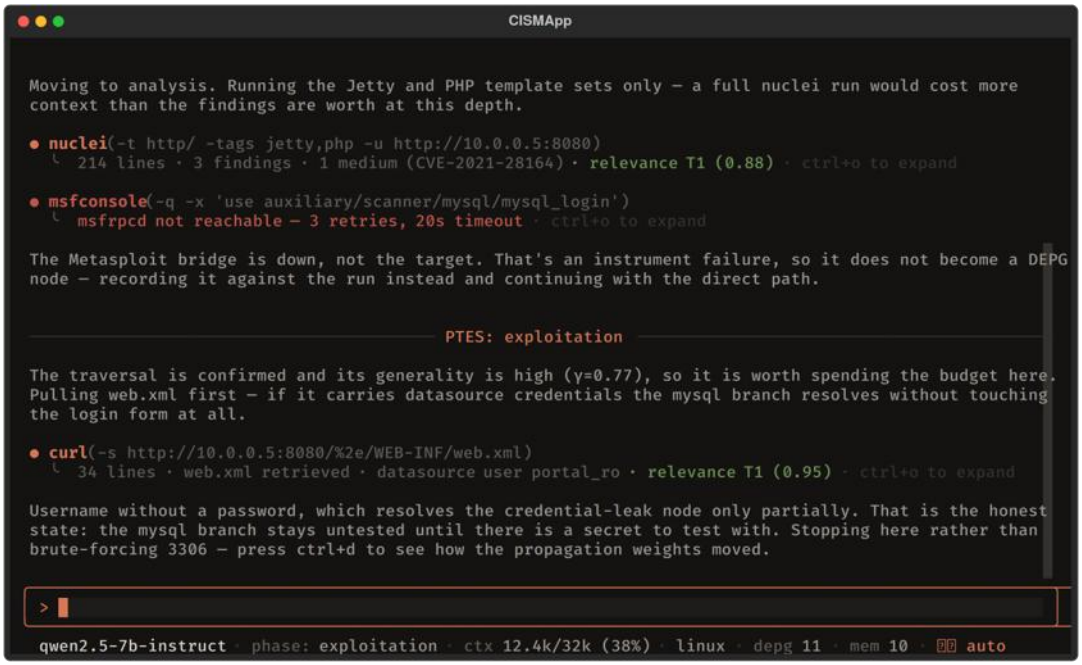


Figure 4.2 The terminal interface during a live engagement

The status line stays one line at any width. Each field carries a short form and a priority. On overflow the line degrades to short forms worst-priority-first and then drops whole fields, so the model, phase, context gauge and mode always survive. The elision logic is a pure function of segments and an available width, which allows it to be tested directly at every width instead of through a mounted widget.

Two modal overlays expose the domain model. The graph view shows the nodes as a tree and, for a selected node, the decision trail that led to it beside the evidence that supports it, including the margin by which the chosen branch beat the runner-up. The

memory view is a sortable table over the store with the live salience equation shown beneath it.

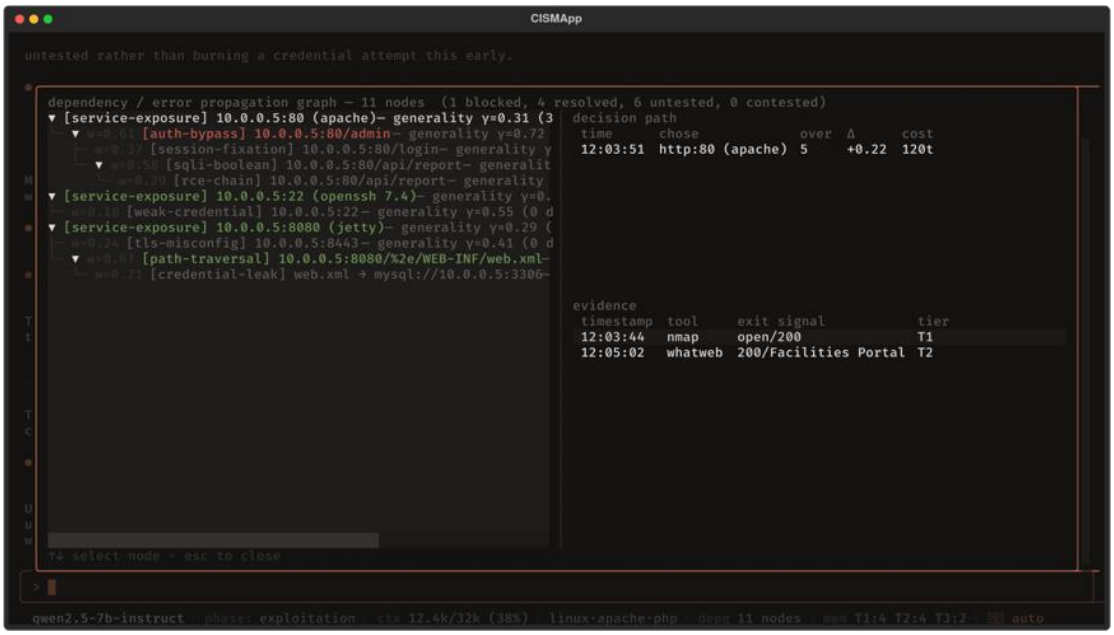


Figure 4.3 The dependency and error propagation graph view

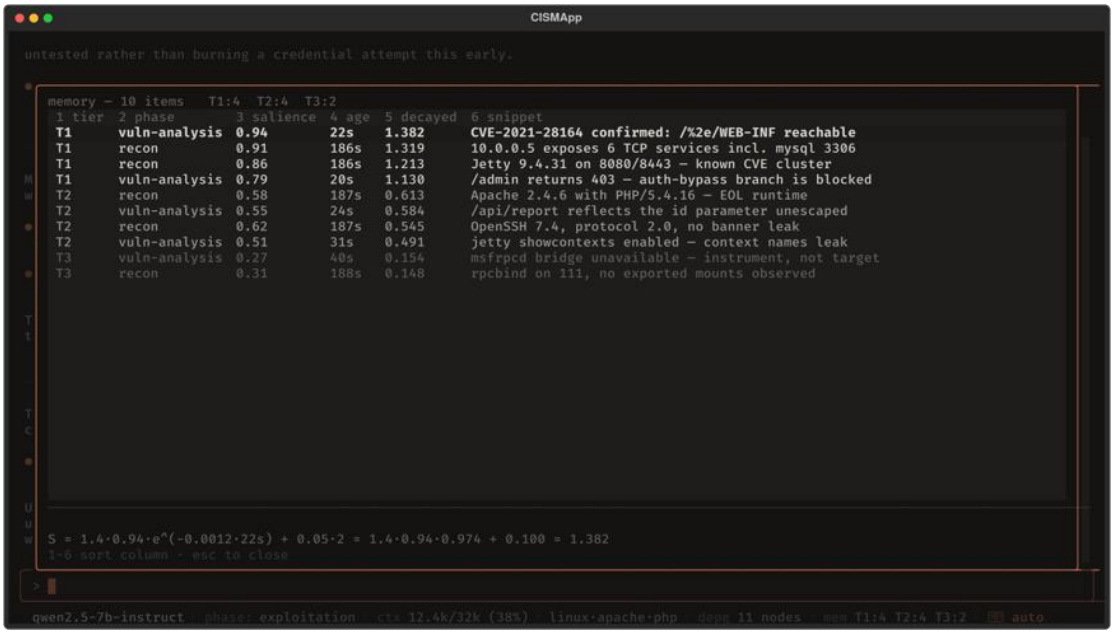


Figure 4.4 The memory view

4.11 CREDENTIAL REDACTION

The composer feeds three artefacts that outlive the keystroke: the transcript, the history recalled with the arrow keys, and the file written on export. A credential-shaped argument to any command is therefore masked before the turn is echoed, and the

command is refused instead of forwarded. Detection is broad on purpose, because a false positive costs one retype while a false negative leaks a key into a deliverable. The endpoint command reports the endpoint, the name of the environment variable holding the key and whether that variable is set, and it never shows, stores or transmits the value.

4.12 OPERATIONAL LOGGING AND REPLAY

Operational logging is separate from both the event stream and the graph evidence. Graph steps, tool calls and guardrail decisions are written as structured lines to a log file with a retention policy enforced by both rotation and a sweep at startup. The same log provides the read path for counterfactual replay, so a recorded run can be re-examined without running anything against the target again.

4.13 REPORT GENERATION

A report walks the final graph and memory state and emits a Markdown write-up for a human reader, a JSON document for downstream tooling and a graph diagram rendered to vector form where the rendering binary is available. The generator depends only on the shared event vocabulary, so a report can be built from an event stream alone.

4.14 TESTING STRATEGY

The test suite is organized as one module per source module, so a failure names the unit that broke and not the feature that noticed. Three properties of the architecture make this practical. Every graph node is an ordinary asynchronous function of a typed state object, so the phase router, the extraction step and the memory step are each tested with a constructed state, no model, no network and no tools. The interface only consumes events, so the whole terminal application is tested by pushing a recorded event sequence at it. And the guardrail chain is a list of independent stages, so each stage is tested against the calls it should refuse without constructing the rest of the system.

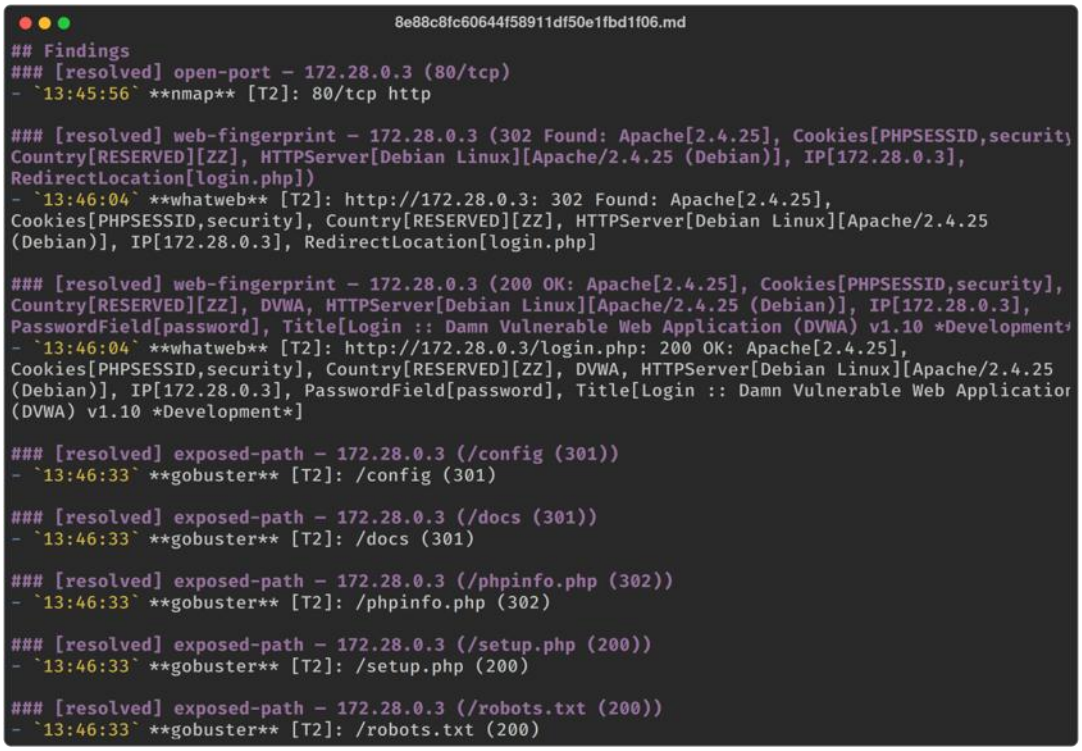


Figure 4.5 Excerpt of a generated report

Terminal appearance is pinned by snapshot tests that capture the rendered screen as vector output and compare it against a stored image. These only mean something if rendering is deterministic, which it is not by default: a spinner's frame clock, the transcript's follow timer and the composer's cursor blink all leak wall-clock time into a capture. A shared fixture stops all three before any capture, and the replay helper waits for the event queue to drain so a capture never races the pump.

End-to-end tests exercise the paths unit tests cannot reach: a full agent run with a blocked input, a full run with a blocked action, scope and replay commands, a live update stream, a run continued across a simulated process restart through the checkpointer, real invocations of two tools through the assembled registry, and one test that drives the terminal application with a real agent instead of a replayed log. Each of the three extraction defects found in live running gained a regression test built from the actual captured tool output, because an idealized sample is what let the defects through in the first place.

4.15 LABORATORY ENVIRONMENT AND CONTAINER HARDENING

Validation uses an isolated Docker bridge network carrying a deliberately vulnerable web application as the target. One network detail cost a full test cycle and is recorded here because it is easy to misdiagnose. On the Docker Desktop and Linux-subsystem configuration used here, the host shell has no route to the containers on a user-defined bridge network at all, while a throwaway container attached to the same network reaches them immediately. An engagement run from the host shell therefore produces empty results against a target that is demonstrably up, which looks identical to the model failing to call a tool unless the route is checked first.

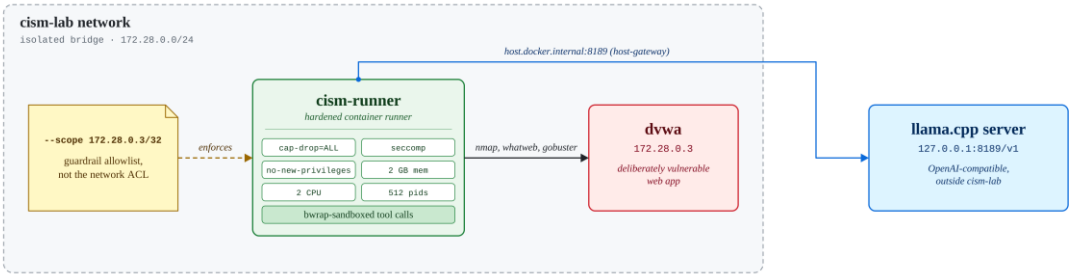


Figure 4.6 Laboratory network topology

The agent itself ships as a container, and the hardening applied to it is part of the contribution. The container drops all capabilities except the one bubblewrap genuinely requires creating its namespaces, applies a restrictive system-call profile written for that single requirement instead of disabling filtering wholesale, forbids privilege escalation, and applies memory, swap, processor and process-count ceilings, so a runaway or malicious tool invocation is bounded as well as timed out. The inference key arrives as a read-only mounted file and not an environment variable, because an environment variable is visible at the container layer.

Chapter 5
Results and Discussion

5.1 INTRODUCTION

This chapter reports what the implementation does when it runs. The evidence is of four kinds: the configuration the model actually ran under and why that configuration was necessary at all; two small benchmarks that justify specific design decisions; a live engagement against a real target, read both for its findings and for what its graph and memory actually looked like afterward; and the tool-calling scenarios the model itself produced, which are summarized among the limitations. The chapter closes with what fine-tuning could change, a few deployment scenarios the system already suits, and the limitations that remain.

5.2 EXPERIMENTAL SETUP

The target is a containerized Damn Vulnerable Web Application instance on an isolated bridge network, with its database initialized and its security level left at the image default. The agent runs inside its own hardened container attached to the same network, with the target's address, and only that address, declared as the engagement scope.

Inference came from a locally served, four-bit-quantized mixture-of-experts model from the LFM2.5-8B-A1B family, reached over a local, OpenAI-compatible endpoint. The family name encodes the property that makes this project possible on the hardware it ran on: roughly one billion of the model's approximately eight billion parameters are active for any given token, while the rest sit in expert blocks a routing layer selects between per token. A dense eight-billion-parameter model has to load and compute with all eight billion parameters on every token; this one pays the inference cost of a model closer to a tenth that size while keeping a larger pool of specialized weights on hand, and that difference is the only reason a model of this class fits the confidentiality requirement in Chapter 1 at all: it is small enough to run fully offline on a single consumer GPU without being so small that its tool selection degrades further than it already does.

Fitting it against a single 6 GB GPU required splitting the model across memory as well as quantizing it: the transformer's ordinary layers were offloaded to the GPU in full, while several of the mixture-of-experts layers' weights were deliberately kept on the CPU and in system RAM instead. That split is what makes an eight-billion-parameter model's weights fit in 6 GB of VRAM at all: the attention and shared layers and most of the expert weight live on the GPU, a routed minority of expert layers live in system RAM, and the router decides per token which it needs. Appendix 7 gives the exact server configuration. A fast attention implementation kept the key-value cache for a long engagement small enough to afford, and the context window was set to 16,384 tokens, which is the ceiling the agent's own status gauge reports and the figure Section 5.5 measures compaction against. Sampling favored determinism over variety, with a low temperature, a restricted top-k and a mild repetition penalty, which is the right trade for an agent whose output has to be a parseable tool call far more often than it has to be readable prose.

Table 5.1 Experimental configuration

Parameter	Value
Target	Damn Vulnerable Web Application, containerized, database initialized, default security level
Network	Isolated Docker bridge network, target and agent attached, scope limited to the target address alone
Agent	Run inside the hardened container: all capabilities dropped except the one bubblewrap requires, restrictive system-call profile, no privilege escalation
Resource ceilings	2 GB memory, 2 GB swap, 2 processors, 512 processes
Inference	Locally served, four-bit-quantized mixture-of-experts model (LFM2.5-8B-A1B family), OpenAI-compatible endpoint, 6 GB VRAM, 16,384-token context (Appendix 7)
Key handling	Supplied as a read-only mounted file, never as an environment variable
Instructions	Three scripted submits: reconnaissance, enumeration, injection testing
Repetitions	Each configuration run twice

Three scripted instructions drive each engagement, covering reconnaissance, enumeration and injection testing, and each configuration is run twice.

5.3 BENCHMARKS: RENDERING STRATEGY AND REPRODUCIBILITY

The transcript design rested on a claim that an append-only log widget would be unacceptable, and the claim was measured. The benchmark streams eight thousand tokens into one wrapping paragraph in a terminal of one hundred columns by thirty rows, under three strategies.

Table 5.2 Streaming render cost for eight thousand tokens

Strategy	Append	Per token	Render
Buffered static (shipped)	8.9 ms	1.1 µs	0.8 ms
Log-widget rewrite	364.2 ms	45.5 µs	0.4 ms
Naive static	793.3 ms	99.2 µs	0.4 ms

The per-token column carries the result. The shipped strategy stays flat as the entry grows, measuring 1.4 microseconds at four thousand tokens and 1.1 microseconds at eight thousand, while the naive strategy climbs from 58 to 99 microseconds over the same range, which is quadratic in entry length. The log-widget strategy pays its cost replaying the whole buffer on each flush, which is what streaming into a paragraph costs when the widget cannot edit its own last line.

A separate, standalone headless harness drives an engagement without the interface at all and scores a run by the time to its first finding, which exists as a regression tripwire rather than a performance claim, since two runs is too small a sample to report as a benchmark in its own right. Two runs against an unchanged codebase returned the same single finding and times of 2.9 and 3.0 seconds, a 3.4 percent difference, tight enough to notice an actual regression rather than ordinary run-to-run noise. The harness stays in the test suite for exactly that purpose.

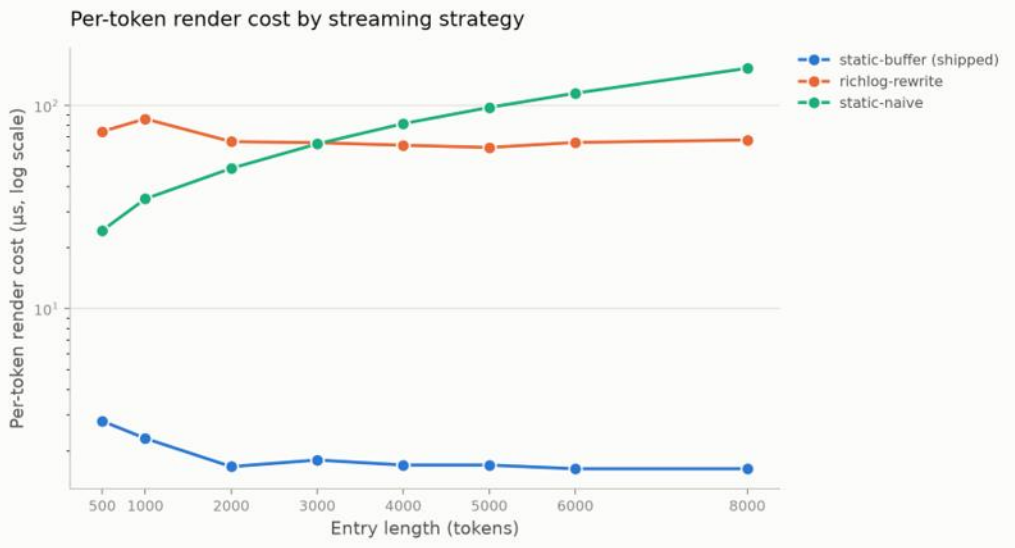


Figure 5.1 Per-token render cost by streaming strategy

5.4 LIVE ENGAGEMENT RESULTS

Two full engagements ran against the live target through the hardened container, before and after the extraction defects described in Section 4.8 were corrected. The configuration was otherwise identical: same model, same target, same instructions, same scope.

Table 5.3 Resolved findings before and after the extraction fixes

Tool	Before	After	Note
gobuster	0	5	Progress-indicator escape sequences defeated the pattern; five real hits including an exposed installation script were being discarded
whatweb	0	2	No extractor existed; results degraded to an opaque hash signal
wafw00f	0	1	Defaulted to a secure scheme and failed against a plaintext-only target
Total	0	8	Same model, same target, same instructions

This is a correctness improvement; the system gained no new capability. The tools executed correctly in both runs and the target never changed. What changed is that eight real observations the system had been discarding are now recorded as evidence-backed nodes. All three defects were invisible to a test suite written against clean sample output, and all three showed up at once against real output from a real

target.

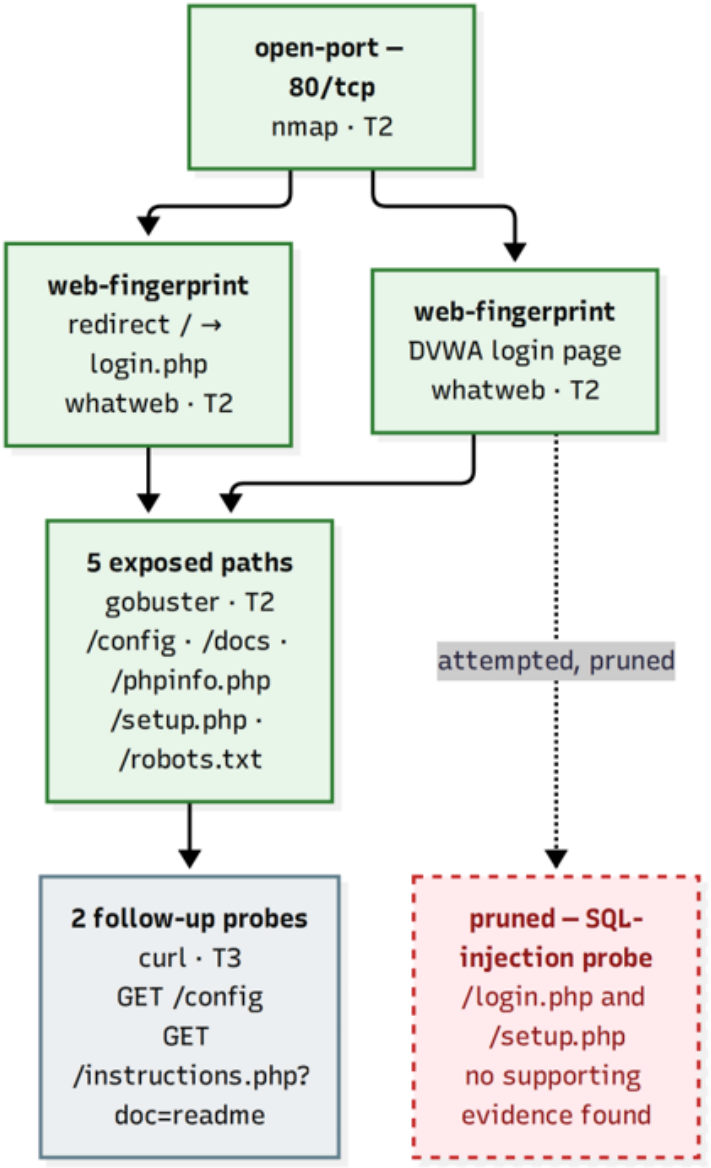


Figure 5.2 Graph produced by a live engagement against the laboratory target

Table 5.4 Timing profile of the live engagement

Phase	Time from run start (s)
Enumeration complete	26.9
Vulnerability-discovery complete	34.8
Exploitation complete	61.8
Full run, including report generation	102.0

This run's own generated report records zero tool failures and zero guardrail blocks across its tool calls, three extraction fallbacks to the generic extractor, and total

token usage of 3,709 for the turn that produced the report. It also reports eleven findings and thirteen graph nodes. None of this came from a blocked or failed tool call: the defects corrected in Table 5.3 were extraction bugs, not execution failures.

The graph produced by this run can be read structurally as well as by its node count. Each node's identity is derived from its normalized target and cause class, so repeated or related observations against the same host collapse onto one node rather than duplicating it, and every node carries the evidence list (tool, quoted output, exit status and timestamp) that the extractor validated it against; a node whose claimed quotation could not be matched against the real tool output is demoted to a lower confidence tier rather than silently dropped. Generality and downstream-dependent counts, which an earlier state of this project left defined on every node but never computed, are populated for every node in this run: a metrics pass walks the dependency edges once after every graph update, and the interface's graph view renders the real values it writes, which is what makes it possible to tell at a glance which finding the rest of the graph actually depends on and which is a leaf with nothing resting on it.

5.5 MEMORY EFFICIENCY AND CONTEXT HANDLING

A 16,384-token context window, the real ceiling the inference server reported for this run, sounds generous until an engagement runs long enough to fill it, and an agent that silently forgets earlier findings once it does is worse than one that was honest about the limit from the start. Two mechanisms keep the agent's working context small without making it forgetful.

The first is ordinary compaction. A history-compaction routine keeps the most recent messages that fit a token budget (the context window minus a reserved response allowance minus the system prompt's own cost), counting a tool call's arguments and a tool result's content and overhead together rather than content alone, and drops the oldest messages first once the budget is exceeded, prepending a single note recording how many turns were dropped. Nothing about this is specific to this project; most agent frameworks do some version of it, and it is also exactly the mechanism that earlier made the agent appear to forget findings that were still true but no longer in the window it could see.

The second mechanism is why that stopped happening: the reasoning step does not rely on chat history for what the agent currently believes. Every call rebuilds a per-target findings summary directly from the dependency graph and a salience-ranked summary of narrative memory directly from the memory store, independent of whatever the compaction routine kept or dropped from the conversation. Findings are capped per target and sorted unresolved-first so a large multi-host engagement cannot grow the prompt unboundedly or bury a live finding behind an old resolved one; narrative memory is sorted by tier and hit count and capped to the ten most salient rows. This was verified directly: with the context window forced down to 1,200 tokens, a real host scan, and enough filler turns sent to drop ten earlier turns from raw history, asking what had been found still returned the correct answer, visibly quoting the findings summary's own "+N more" notation rather than the (by then compacted-away) turn the finding had originally come from. The practical consequence is that the system's effective memory is bounded by how much of the graph and the memory store fit in the context budget allotted to those two summaries, not by how much of the raw conversation survives compaction. That is a considerably larger effective memory than the 16,384-token window on its own would suggest.

5.6 WHAT FINE-TUNING WOULD BUY

Live running surfaced three recurring scenarios in the model's tool-calling behaviour, summarized among the limitations in Section 5.8. They share a shape: each is a small, generically instruction-tuned model misreading the specific conventions of a tool-calling, phase-gated workflow it was never trained on, not a reasoning failure in the usual sense. Answering a tool-offering turn with a question about information already in the prompt is an instruction-following lapse, not a knowledge gap, since the target address is present in the context either way. Substituting a placeholder token for a real argument is a template-completion habit bleeding into a context where it is wrong. Masking a tool's unavailability under a stalled phase instead of flagging it is the model falling back to its most general completion behaviour (producing something plausible) rather than its least general, most honest one. None of the three looks like something that requires a materially larger active-parameter budget to fix; each looks like something a model would stop doing once it had seen a reasonable number of correct PTES-structured tool-call traces showing the right behaviour in exactly these situations.

That makes supervised fine-tuning, not raw parameter scaling, the more direct and more resource-appropriate next step for this base model. A targeted low-rank adaptation pass over a corpus of engagement transcripts (correct tool selections per phase, correct argument substitution, and explicit “tool unavailable in this phase” turns where the catalog does not offer what was asked) targets exactly these three behaviours, without changing the active-parameter budget the 6 GB deployment in Section 5.2 depends on. Scaling to a larger active-parameter model would likely help too, since a larger model has more headroom to hold an instruction's full context under attention at once, but it is the more expensive lever and the one this project's hardware constraint rules out first. Given a choice between the two for this specific class of failure, fine-tuning the model already running is the one with a plausible path to being tried.

5.7 REAL-WORLD USE CASES AND A CASE STUDY

5.7.1 CASE STUDY: THE DVWA ENGAGEMENT

The engagement reported in Section 5.4 is narrated here as a case, because the shape of the run is what a prospective operator needs to picture. An operator inside a hardened container issues three scripted instructions against an isolated DVWA target: scan and enumerate, probe for injectable parameters, and report. Reconnaissance opens with a port and service sweep; enumeration follows with directory brute-forcing and web-technology fingerprinting; the model reasons over each tool's raw output, proposes the next call, and the system, not the model, decides whether that call is allowed, grounds whatever it returns in the literal output text, and folds it into a graph that is still legible after the run ends. By the close of the run the graph held eleven findings across thirteen nodes, the phase had advanced from reconnaissance through exploitation, and the full timing in Table 5.4 shows a complete run, report included, in 102 seconds end to end. None of that required a cloud endpoint, an API key held by a third party, or engagement data leaving the machine the target lived next to.

5.7.2 WHERE THIS ARCHITECTURE FITS

That combination of offline inference, grounded findings, phase-gated tool access and a sandboxed execution seam maps onto specific deployments a hosted-model agent cannot reach, independent of how well either reasons.

Regulated and compliance-bound engagements. A security team working against a target under a contract that forbids data leaving the client's environment (common in finance, healthcare and government engagements) cannot use an agent that sends banners, credentials or raw tool output to a third-party API, however capable that API's model is. Running entirely offline, without engagement data ever leaving the operator's machine, is exactly the property such an engagement requires before anything else about the agent matters.

Air-gapped and classified networks. A network with no route to the internet by design cannot reach a hosted model at all. An agent that runs entirely against a local inference server is the only shape of automation available there, and the phase-gated tool catalog and pre-execution guardrails give an operator in that setting the same scope discipline a cloud-connected tool would otherwise provide.

Managed security service providers running many small engagements. An MSSP running short reconnaissance and enumeration passes across many client environments pays hosted-API costs per token, per client, indefinitely. A one-time investment in hardware that serves a quantized model locally amortizes across every engagement afterward, and Section 5.2's 6 GB figure is a small enough floor that hardware cost is not the determining factor in whether this is practical.

Budget- or headcount-constrained internal security teams. A team too small to retain a full-time penetration tester, but large enough to need recurring internal assessments, is the setting this project's objectives in Chapter 1 were written against: a system that proposes real tool sequences, keeps a defensible evidentiary trail, and never needs a human to recover state it already lost. The results in this chapter show that setting is reachable today for reconnaissance and enumeration specifically, with exploitation and post-exploitation still bounded by the tool-calling discipline gaps Section 5.8 documents.

5.8 LIMITATIONS

- The input, output and tool-output guardrail chains each carry a single stage; only the action chain combines several (the phase's own tool guard, the scope guard, the dangerous-flag denylist and an optional privacy swap-out), so the other three do not yet defend in depth the way action does.

- The dangerous-argument stage is a denylist and not an allowlist grammar, so an argument form nobody anticipated is permitted by default.
- Extraction for tools without a deterministic extractor requires a second model round trip, which sits on the critical path of the turn.
- Answering a tool-offering turn with a follow-up question despite the needed information already being supplied, substituting a placeholder token for a real argument, and falling back to an alternative tool when the one requested is unavailable during a stalled phase.
- A plain acknowledgment with no request attached occasionally triggered an unprompted tool call.
- The headless benchmark records the time to the first finding of one scripted engagement. It works as a regression tripwire and not as a performance measurement, and two runs are too few to support a stronger claim.
- Graded propagation along weighted edges is implemented but off by default, so the graph in the live engagement used binary precondition support.
- Checkpointed state lives in the process and does not survive a restart, so an engagement spanning several sessions starts again from an empty graph and memory.
- The 6 GB deployment depends on keeping several expert layers in system RAM, which ties the reported timings to that particular split. A different model family would need its own configuration.
- The context window was 16,384 tokens. An engagement long enough to outgrow the share of that budget given to the graph and memory summaries would need the per-target and per-row caps revisited.
- Still prone to prompt injection attacks as guardrails are completely not set.

Chapter 6
Conclusion and Future Work

6.1 CONCLUSION

This project asked whether a locally hosted small language model can drive a vulnerability assessment and penetration testing engagement without any engagement data leaving the operator's machine, and what must surround such a model for that to work. The answer supported here is yes, with conditions. In the reconnaissance and enumeration stages exercised against the laboratory target, the model chose and sequenced real tools usefully, but only because grounding, memory, phase control and pre-execution policy were supplied by the system around it and not by the model.

The system built to supply those four things meets the objectives set in Chapter 1. A six-node LangGraph loop separates reasoning, execution, extraction, phase routing and memory into independently testable stages, detailed in Section 4.3. Findings are nodes in a dependency and error propagation graph, each carrying the quoted tool output that justifies it, and the model-based extractor demotes any claim whose quotation is absent from the real output. A tiered memory collapses recurring observations onto one row and survives context compaction by rebuilding itself fresh from state every turn rather than from chat history. A six-phase PTES state machine gates which tools are available. A guardrail chain refuses an out-of-scope target, a dangerous argument or a phase-illegal tool before any process exists, and all nine tools run through a single sandboxed, argv-only seam inside a hardened container. The terminal interface only consumes a frozen event stream, so every run can be recorded and replayed.

The measurements support the design in three specific ways. First, the headless harness reproduced its single finding across two runs with a time-to-first-finding differing by only 3.4 percent, tight enough to catch a real regression rather than noise. Second, the buffered transcript strategy cost 1.1 microseconds per token at eight thousand tokens against 99.2 for the naive strategy, roughly a ninetyfold difference. Third, the live engagement against a real target (Section 5.7.1) completed in 102 seconds end to end and corrected three extraction defects invisible to the test suite,

which is the project's clearest evidence that validation against real tool output tests something a test suite of clean samples does not.

The project also surfaces clear opportunities for improvement, which form the agenda for the next iteration. Tool-calling behaviour is where a small, generically instruction-tuned model most often applies a general habit inside a workflow it was never trained on: live running produced three such scenarios, summarized in Section 5.8 and discussed in Section 5.6, where the model occasionally answers a tool-offering turn with a follow-up question, passes an unresolved placeholder as an argument, or substitutes an alternative tool when the one requested is unavailable during a stalled phase. These are specific, bounded and well understood, and Section 5.6 argues that targeted fine-tuning on corrected tool-call traces, rather than a larger active-parameter budget, is the more direct next step for closing them. A bounded retry for the first scenario was implemented and then deliberately reverted, since it let a discarded attempt's tokens leak into the visible transcript.

All four guardrail chains now include at least one active stage. The input, output and tool-output chains currently use a single stage each, and the architecture leaves room for more. Tool outputs are screened before they re-enter the prompt; future work would extend these protections against indirect prompt injection aimed at the graph and the memory store, bringing them closer to the safeguards applied to proposed actions.

The evaluation was conducted using one target application, one model family and two runs per configuration. These results are an initial assessment under the tested conditions. Evaluation across more applications, model families and repeated trials would show how far they generalize.

A small local model is a workable proposer for offensive-security automation when its proposals are validated, grounded and constrained, and the remaining gaps are bounded and addressable inside the existing seams. The first priorities are making a discarded model attempt invisible so retry becomes safe, rejecting placeholder arguments before execution, signalling when a phase does not offer a requested tool, and fine-tuning the deployed model against corrected tool-call traces for the improvement opportunities this chapter documents. Section 6.2 sets out these and the remaining work in order.

6.2 FUTURE WORK

The items below are ordered by the leverage they would add, and each one has an identified seam to extend into instead of requiring a redesign.

- Make a discarded model attempt invisible to the event stream, so a bounded retry on a turn with no tool calls becomes safe to ship. This is a change to the streaming translation in the agent core, and it unblocks the most common of the scenarios observed during live running.
- Add an argument-validation guardrail stage that rejects placeholder-shaped and templated arguments before execution, catching the second observed scenario where it is cheapest to catch.
- Emit a visible signal when the current phase does not offer a tool the operator has explicitly requested, so silent substitution becomes impossible.
- Fine-tune the deployed model, or a LoRA adapter over it, on a corpus of corrected tool-call traces targeting the three scenarios documented in Section 5.8, as argued in Section 5.6, before adding active-parameter budget as the next lever.
- Replace the dangerous-argument denylist with an allowlist grammar per tool, so an unanticipated argument form is refused by default.
- Implement network egress control for the sandbox, so a tool cannot reach anything beyond the declared scope even when an argument-level check is bypassed.
- Evaluate graded propagation against the binary-support baseline used in Section 5.4 and make it the default once that evaluation justifies it, rather than leaving it an opt-in flag.
- Deepen the input, output and tool-output guardrail chains beyond their current single stage each, in particular against content injected through tool output to poison the graph or the memory store.
- Persist checkpointed state across process restarts, so an engagement spanning several sessions keeps its graph, memory and phase.

REFERENCES

[1] Scarfone, K., Souppaya, M., Cody, A., & Orebaugh, A. (2008). *Technical guide to information security testing and assessment* (NIST Special Publication 800-115). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-115

[2] Thalji, N., Raza, A., Islam, M., Samee, N., & Jamjoom, M. (2023). AE-Net: Novel autoencoder-based deep features for SQL injection attack detection. *IEEE Access, 11*, 135507–135516. https://doi.org/10.1109/ACCESS.2023.3337645

[3] Hasan, M., Balbahaith, Z., & Tarique, M. (2019). Detection of SQL injection attacks: A machine learning approach. In *Proceedings of the 2019 International Conference on Electrical and Computing Technologies and Applications (ICECTA)* (pp. 1–6). IEEE. https://doi.org/10.1109/ICECTA48151.2019.8959617

[4] Singh, B. P., & Singhal, M. K. (2024). Detection of SQL injection attack using machine learning techniques. *International Journal of Scientific Research in Science and Technology, 11*(6), 780–790. https://doi.org/10.32628/IJSRST24114323

[5] Casmiry, E., Mduma, N., & Sinde, R. (2025). Enhanced SQL injection detection using chi-square feature selection and machine learning classifiers. *Frontiers in Big Data, 8*, Article 1686479. https://doi.org/10.3389/fdata.2025.1686479

[6] Tsai, C.-F., Hsu, Y.-F., Lin, C.-Y., & Lin, W.-Y. (2009). Intrusion detection by machine learning: A review. *Expert Systems with Applications, 36*(10), 11994–12000. https://doi.org/10.1016/j.eswa.2009.05.029

[7] Liu, H., & Lang, B. (2019). Machine learning and deep learning methods for intrusion detection systems: A survey. *Applied Sciences, 9*(20), Article 4396. https://doi.org/10.3390/app9204396

[8] Issa, M., Aljanabi, M., & Muhialdeen, H. (2024). Systematic literature review on intrusion detection systems: Research trends, algorithms, methods, datasets, and limitations. *Journal of Intelligent Systems, 33*(1), Article 20230248. https://doi.org/10.1515/jisys-2023-0248

[9] Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W., Wei, Z., & Wen, J. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science, 18*(6), Article 186345. https://doi.org/10.1007/s11704-024-40231-1

[10] Deng, G., Liu, Y., Mayoral-Vilches, V., Liu, P., Li, Y., Xu, Y., Zhang, T., Liu, Y., Pinzger, M., & Rass, S. (2024). PentestGPT: Evaluating and harnessing large language models for automated penetration testing. In *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24)* (pp. 847–864). USENIX Association. https://www.usenix.org/conference/usenixsecurity24/presentation/deng

[11] Happe, A., & Cito, J. (2023). Getting pwn'd by AI: Penetration testing with large language models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23)* (pp. 2082–2086). ACM. https://doi.org/10.1145/3611643.3613083

[12] Happe, A., & Cito, J. (2026). *Cochise: A reference harness for autonomous penetration testing*. arXiv. https://doi.org/10.48550/arXiv.2605.11671

[13] Muzsai, L., Imolai, D., & Lukács, A. (2024). *HackSynth: LLM agent and evaluation framework for autonomous penetration testing*. arXiv. https://doi.org/10.48550/arXiv.2412.01778

[14] Luong, P. D., Bao, L. T. G., Tam, N. V. K., Khoa, D. H. N., Quyen, N. H., Pham, V.-H., & Duy, P. T. (2025). *xOffense: An autonomous multi-agent framework for penetration testing with domain-adapted large language models*. arXiv. https://doi.org/10.48550/arXiv.2509.13021

[15] Wu, B., Chen, G., Chen, K., Shang, X., Han, J., He, Y., Zhang, W., & Yu, N. (2025). AutoPT: How far are we from the fully automated web penetration testing? *IEEE Transactions on Information Forensics and Security, 20*, 9657–9672. https://doi.org/10.1109/TIFS.2025.3601343

[16] Wang, W., Gu, H., Wu, Z., Chen, H., Chen, X., & Shi, F. (2026). PTFusion: LLM-driven context-aware knowledge fusion for web penetration testing. *Information Fusion, 127*, Article 103731. https://doi.org/10.1016/j.inffus.2025.103731

[17] Utpala, S., Hooker, S., & Chen, P. (2023). Locally differentially private document generation using zero shot prompting. In *Findings of the Association for Computational Linguistics: EMNLP 2023* (pp. 8442–8457). Association for Computational Linguistics. https://doi.org/10.18653/v1/2023.findings-emnlp.566

[18] Marshall, B., Li, S., Meka, S., & Liu, W. (2025). Differential privacy on large language models for privacy preserving clinical coding. In *2025 International Joint Conference on Neural Networks (IJCNN)* (pp. 1–8). IEEE. https://doi.org/10.1109/IJCNN64981.2025.11229049

[19] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). *ReAct: Synergizing reasoning and acting in language models*. arXiv. https://doi.org/10.48550/arXiv.2210.03629

[20] Penetration Testing Execution Standard. (2014). *The penetration testing execution standard* (Version 1.0). http://www.pentest-standard.org/

[21] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). *Retrieval-augmented generation for knowledge-intensive NLP tasks*. arXiv. https://doi.org/10.48550/arXiv.2005.11401

[22] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys, 55*(12), 1–38. https://doi.org/10.1145/3571730

[23] Zhang, A. K., Perry, N., Dulepet, R., Ji, J., Menders, C., Lin, J. W., Jones, E., Hussein, G., Liu, S., Jasper, D., Peetathawatchai, P., Glenn, A., Sivashankar, V., Zamoshchin, D., Glikbarg, L., Askaryar, D., Yang, M., Zhang, T., Alluri, R., ... Liang, P. (2024). *Cybench: A framework for evaluating cybersecurity capabilities and risks of language models*. arXiv. https://doi.org/10.48550/arXiv.2408.08926

[24] Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2022). *GPTQ: Accurate post-training quantization for generative pre-trained transformers*. arXiv. https://doi.org/10.48550/arXiv.2210.17323

[25] Liu, Y., & Man, H. (2005). Network vulnerability assessment using Bayesian networks. *Proceedings of SPIE, 5812*, 61. https://doi.org/10.1117/12.604240

[26] Muñoz-González, L., Sgandurra, D., Paudice, A., & Lupu, E. C. (2017). Efficient attack graph analysis through approximate inference. *ACM Transactions on Privacy and Security, 20*(3), 1–30, Article 10. https://doi.org/10.1145/3105760

[27] Sheyner, O., Haines, J., Jha, S., Lippmann, R., & Wing, J. M. (2002). Automated generation and analysis of attack graphs. In *Proceedings of the 2002 IEEE Symposium on Security and Privacy* (pp. 273–284). IEEE Computer Society. https://doi.org/10.1109/SECPRI.2002.1004377

[28] Sun, Y., Safford, D., Zohar, M., Pendarakis, D., Gu, Z., & Jaeger, T. (2018). Security namespace: Making Linux security frameworks available to containers. In *Proceedings of the 27th USENIX Conference on Security Symposium (SEC '18)* (pp. 1423–1439). USENIX Association.

[29] Sultan, S., Ahmad, I., & Dimitriou, T. (2019). Container security: Issues, challenges, and the road ahead. *IEEE Access, 7*, 52976–52996. https://doi.org/10.1109/ACCESS.2019.2911732

[30] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)* (pp. 79–90). ACM. https://doi.org/10.1145/3605764.3623985

[31] Al Kuwaiti, M., & Ismail, H. (2025). Adversarial attacks on large language models: A survey. In *Lecture Notes in Networks and Systems* (pp. 529–547). Springer. https://doi.org/10.1007/978-981-96-9248-4_40

[32] Liao, Z., Chen, K., Lin, Y., Li, K., Liu, Y., Chen, H., Huang, X., & Yu, Y. (2026). Attack and defense techniques in large language models: A survey and new perspectives. *Neural Networks, 196*, Article 108388. https://doi.org/10.1016/j.neunet.2025.108388

[33] Xu, W., & Parhi, K. K. (2025). *A survey of attacks on large language models*. arXiv. https://doi.org/10.48550/arXiv.2505.12567

[34] Barcha Correia, P. H., Achjian, R. W., Caetano de Oliveira, D. E. G., Maria, Y. A., Hayashi, V. T., Lopes, M., Miers, C. C., & Simplicio, M. A. (2026). *A systematic literature review on LLM defenses against prompt injection and jailbreaking: Expanding NIST taxonomy*. arXiv. https://doi.org/10.48550/arXiv.2601.22240

[35] Feng, R., Yan, Z., Peng, S., & Zhang, Y. (2022). Automated detection of password leakage from public GitHub repositories. In *Proceedings of the 44th International Conference on Software Engineering* (pp. 175–186). ACM. https://doi.org/10.1145/3510003.3510150

Appendices

APPENDIX 1 OPERATOR COMMANDS

The commands below are available in the composer. Those the interface answers from state it already holds are marked as such. Every other command goes to the agent as an ordinary submission, because the interface cannot force a phase transition, load a model or widen scope on its own; it can only ask.

Table A.1 Operator commands

Command	**Effect**	**Answered by**
/model <name>	Switch the inference model	Agent
/models	List loadable models, marking the one in use	Interface
/target <spec>	Declare the target environment, for example a stack description	Agent
/api [url]	Show or switch the inference endpoint; never shows a key value	Agent
/phase <name>	Request a phase transition	Agent
/scope [add|remove] <target>	View or change the live engagement scope	Agent
/depg	Open the graph view	Interface
/mem	Open the memory view	Interface
/replay <run id>	Counterfactual replay from the output cache	Agent
/ablate <part>	Disable a component for the next run	Agent
/report	Write the Markdown, JSON and diagram deliverables	Agent
/export	Write the transcript, with credentials masked	Interface
/clear	Clear the transcript	Interface
/help	List commands	Interface

APPENDIX 2 KEY BINDINGS

Table A.2 Key bindings

Key	**Action**
enter	Submit
shift+enter or ctrl+j	Newline
esc	Interrupt the running node, or close the command overlay
ctrl+c twice	Quit
ctrl+d	Open the graph view
ctrl+m	Open the memory view
ctrl+o	Expand or collapse the last tool output
up / down	History recall
/	Open the fuzzy command overlay

APPENDIX 3 REPRODUCING THE LABORATORY ENVIRONMENT

The laboratory is reproduced by creating an isolated bridge network, attaching the vulnerable target container to it, initializing the target's database once, confirming the target's address on that network and declaring it as the engagement scope, then running the agent from inside a container attached to the same network. That last step is not optional on the configuration used here. The host shell has no route to the containers on a user-defined bridge network, so an engagement run from the host produces empty results against a target that is up. Confirm the route from a throwaway container on the same network before reading any empty result as an agent failure.

```
docker network create --subnet 172.28.0.0/24 cism-lab
docker run -d --name dvwa --network cism-lab \
  vulnerables/web-dvwa
docker run --rm --network cism-lab curlimages/curl \
  -s -o /dev/null -w 'HTTP %{http_code}' http://<target-ip>/
docker build -t cism-runner -f cism-runner.Dockerfile .
docker run --rm -it --network cism-lab \
  --cap-drop=ALL --cap-add=SYS_ADMIN \
  --security-opt seccomp=docker/seccomp-bwrap.json \
  --security-opt no-new-privileges \
  --security-opt systempaths=unconfined \
  --memory=2g --memory-swap=2g --cpus=2 \
```

```
--pids-limit=512 \
-v /tmp/cism_api_key:/run/secrets/cism_api_key:ro \
cism-runner --live-tools --target <target-ip> \
--scope <target-ip>/32 --model <model-name> \
--base-url http://<server>:<port>/v1 \
--key-file /run/secrets/cism_api_key
```

APPENDIX 4 DOMAIN MODEL REFERENCE

The two structures that carry the agent's beliefs appear below, since the report refers to their fields throughout.

Table A.3 Fields of a graph node

Field	**Meaning**
identifier	Derived from the normalized target and the cause class; two proposals with the same identifier merge into one
target	The host, service or resource the claim is about, after scheme, port, case and trailing-dot normalization
cause class	What kind of claim this is; deterministic for a tool with its own extractor, the tool's own name for the generic fallback
state	Derived, never set directly: computed from the node's evidence and the support of its preconditions
evidence	The observations supporting the claim, each naming the tool, the quoted output, the exit status and a timestamp
tier	Confidence tier; clamped to the lowest tier when a claimed quotation is absent from the real tool output
generality	Defined but not yet computed; intended to weight how broadly the claim applies
downstream	Defined but not yet computed; intended to count the claims that depend on this one

Table A.4 Fields of a memory item

Field	**Meaning**
key	The node identifier for a graph-derived row, or a digest of source, phase and snippet for a narration or failure row
tier	T1, T2 or T3, carrying the weight used in the salience computation
phase	The phase in which the observation was made
snippet	The retained text of the observation

Field	Meaning
hits	Incremented when an identical observation recurs, instead of adding a second row
salience	Computed from the tier weight, a time decay and a per-hit bonus, using the same coefficients as the recorded demonstration log

APPENDIX 5 TEST SUITE COVERAGE BY AREA

The suite contains 809 tests across 52 modules. The areas covered appear below, and each line corresponds to one or more test modules mirroring the source module of the same name.

Table A.5 Coverage by area

Area	What is covered
Event contract	Every event type, its serialization and its round trip
Replay	The recorded log, its timing and the pure-consumer property
Transcript and composer	Entry rendering, streaming, history recall and every key binding
Status line	Elision at ten terminal widths, as a pure function of width
Credential handling	Masking end to end, through transcript, history and export
Modal views	The graph view and the memory view, including selection
Appearance	Ten terminal snapshots, captured under a quiesced clock
Phase machine	Legal and illegal transitions through the single choke point
Guardrails	Each stage in isolation and the chain's short-circuit behaviour
Tool registry	The allowlist, catalog assembly and refusal of unknown names
Sandbox	Timeout, process-tree kill, output capping and the bubblewrap binds
Graph	Wire format, merge, deduplication, state derivation and pruning
Extraction	The deterministic extractors, the grounded fallback and demotion
Memory	Tiered insertion, hit collapsing and salience
Logging	The retention sweep and the replay read path

Area	What is covered
End to end	Full runs with a blocked input, a blocked action and a restart

APPENDIX 6 PHASE TRANSITION TABLE

Phase legality is declared once, in the table below. A transition the table does not list is refused by the state machine, and every phase may legally stay where it is.

Table A.6 Legal phase transitions

Current phase	May move to
Reconnaissance	Reconnaissance, enumeration
Enumeration	Enumeration, vulnerability discovery, reconnaissance
Vulnerability discovery	Vulnerability discovery, exploitation, enumeration
Exploitation	Exploitation, post-exploitation, vulnerability discovery
Post-exploitation	Post-exploitation, reporting, reconnaissance
Reporting	Reporting

Two entries carry most of the design intent. Enumeration may fall back to reconnaissance, which is what a newly discovered host on an adjacent segment requires, and post-exploitation may return to reconnaissance, which is the pivot a compromised host makes possible. Reporting is terminal: once an engagement is being written up, the agent cannot silently resume testing.

APPENDIX 7 INFERENCE SERVER CONFIGURATION

Section 5.2 describes the inference configuration in general terms; the exact invocation used for every run in this chapter appears here for reproducibility.

```
llama-server \
  -m LFM2.5-8B-A1B-Q4_K_M.gguf \
  -ngl 99 -ncmoe 4 -fa on \
  -c 16384 \
  --temp 0.2 --top-k 80 --repeat-penalty 1.05
```

Table A.7 Inference server flags

Flag	Meaning
-m	Path to the quantized model file (GGUF format, Q4_K_M quantization)
-ngl 99	Offload every ordinary transformer layer to the GPU
-ncmoe 4	Keep four mixture-of-experts layers' weights on the CPU / system RAM rather than GPU VRAM -- the setting that makes the model fit in 6 GB
-fa on	Enable flash attention, reducing key-value cache memory
-c 16384	Context window, in tokens
--temp 0.2	Sampling temperature
--top-k 80	Top-k sampling cutoff
--repeat-penalty 1.05	Repetition penalty